National University of Computer and Emerging Sciences, Lahore

VibePen

Esar Fatima 23L-0888 BS(CS)

Haleemah Zaheer 23L-0554 BS(CS)

Haniya Farooq 23L-2650 BS(CS)

Supervisor: Arslan Asif

Final Year Project

October 10, 2026

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: VibePen

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: ..10/10/2026.....

Name: .Haleemah Zaheer.....

Signature: .....

Name: .Haniya Farooq.....

Signature: .....

Name: ...Esar Fatima.....

Signature: .....

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

“Vibe coding” enables non-technical users and developers to create full-stack web applications by describing their requirements in natural language to platforms such as Lovable, Bolt, Replit, and Base44. Although this approach accelerates software development, it can also result in applications being deployed without sufficient security review. Research on AI-generated code has examined security weak-nesses in generated contributions and projects, highlighting the importance of reviewing generated code before deployment. Common web application security risks include inadequate access control, injection vulnerabilities, and security misconfiguration. These risks highlight the need for accessible security assessment tools that can evaluate applications created through AI-assisted development.

VibePen is a vulnerability-scanning system designed to assess such applications through two input modes: source-code archive analysis and authorized live-website scanning. Its proposed analysis pipeline combines static analysis using Semgrep and Tree-sitter with dynamic analysis through a Playwright-based crawler. A large language model (LLM) reasoning layer is intended to assess candidate findings, correlate evidence across files, and help reduce false positives. The system presents prioritized findings in a plain-language report containing supporting evidence and remediation guidance. To reduce the risk of unauthorized testing, live scanning is restricted to domains whose ownership has been verified through an approved verification mechanism. VibePen is designed to identify and report potential vulnerabilities rather than modify the target application.

This report presents the project background, vision and scope, related tools and research, software requirements, system architecture, and detailed design. It establishes the requirements and design foundation for implementation and subsequent evaluation using authorized benchmark applications and appli-cations generated through AI-assisted development.

Executive Summary

AI-assisted development platforms allow users with limited programming experience to create and deploy web applications rapidly. However, generated code can contain security weaknesses, and conventional web application risks such as inadequate access control, injection, and security misconfiguration remain relevant [1, 2]. Conventional security tools can identify many of these issues, but their technical output and configuration requirements may make them difficult for non-specialists to use effectively. VibePen addresses this problem by providing a security assessment workflow intended for applications created through natural-language prompting and AI-generated code. Its goal is to make potential vulnerabilities easier to identify, understand, prioritize, and address before they lead to security incidents.

VibePen is designed to support source-code and live-website scanning. In source-code mode, users submit a project archive for analysis without executing the submitted application. In live-website mode, users must first demonstrate control of the target domain through an approved verification method. A rate-limited crawler then examines the authorized website. Static and dynamic analysis components identify candidate vulnerabilities, while an LLM-based reasoning layer evaluates the available evidence and helps connect related findings. The system produces a severity-ranked report with explanations, evidence, and recommended remediation steps. VibePen is intended to report potential weaknesses rather than make changes to the target application, and its findings require appropriate interpretation and validation.

This deliverable defines the requirements and design needed to guide implementation. It covers the project background and objectives, related security tools and research, functional and non-functional requirements, use cases, interface designs, database structure, system architecture, module interactions, API design, and security controls. These elements establish a common specification for development and testing. The design also identifies operational constraints and risks, including false positives, incomplete scan coverage, external service dependencies, and the need to prevent unauthorized website testing.

The next project phase will implement the specified components and evaluate their performance against authorized benchmark applications. The evaluation is intended to include open-source applications, intentionally vulnerable applications, and applications generated by the project team using AI-assisted development tools. The resulting evaluation will examine whether the system identifies relevant weaknesses, produces useful evidence, and communicates findings clearly. Implementation details, evaluation results, user documentation, and the final project report will establish the extent to which VibePen meets its objectives and where further improvements are required.

Table of Contents

List of Figures **ix**

List of Tables **xi**

1 Introduction **1**

1.1 Purpose of this Document 1

1.2 Intended Audience 1

1.3 Definitions, Acronyms, and Abbreviations 2

1.4 Conclusion 3

2 Project Vision **4**

2.1 Problem Domain Overview 4

2.2 Problem Statement 4

2.3 Problem Elaboration 5

2.3.1 Security Weaknesses Typical of AI-Generated Applications 5

2.3.2 Limits of a Single Analysis Technique 6

2.3.3 Results That Non-Experts Cannot Use 6

2.3.4 Authorized Testing 6

2.4 Goals and Objectives 7

2.5 Project Scope 7

2.5.1 In Scope 8

2.5.2 Out of Scope 9

2.6 Sustainable Development Goal (SDG) 9

2.7 Constraints 10

2.8 Business Opportunity 10

2.9 Stakeholders Description/ User Characteristics 11

2.9.1 Stakeholders Summary 11

2.9.2 Key High-Level Goals and Problems of Stakeholders 11

2.10 Conclusion 12

3 Related Applications 14

3.1 Introduction 14

3.2 Static Analysis and Source-Code Parsing 14

3.3 Secret and Dependency Scanning 15

3.4 Dynamic Analysis and Browser Automation 16

3.5 AI-Assisted Security Analysis 17

3.6 Evaluation Applications and Test Environments 17

3.7 Comparison of Related Applications 18

3.8 Identified Gap 18

3.9 Positioning of VibePen 20

3.10 Ethical and Legal Considerations 20

3.11 Conclusion 21

4 Software Requirement Specifications 22

4.1 List of Features 22

4.2 Functional Requirements 22

4.3 Quality Attributes and Non-Functional Requirements 24

4.4 Assumptions and Constraints 25

4.5 Use Cases 26

4.5.1 UC-01: Authenticate User 26

4.5.2 UC-02: Submit Code Archive 27

4.5.3 UC-03: Verify Domain Ownership 28

4.5.4 UC-04: Run a Verified Website Scan 29

4.5.5 UC-05: Review and Export Report 30

4.5.6 UC-06: Administer Users and Rules 31

4.6 Use Case Model 31

4.7 Hardware and Software Requirements 32

4.7.1 Hardware requirements 33

4.7.2 Software requirements 33

4.8 Graphical User Interface 33

4.9 Database Design 38

4.9.1 Entity-relationship diagram 38

4.9.2 Data dictionary 39

4.10 Risk Analysis 41

4.11 Conclusion 42

6 High-Level and Low-Level Design **43**

43

6.1 System Overview 43

6.1.1 Design Principles 43

6.1.2 System Architecture 44

6.1.3 Technology Baseline 44

6.1.4 Module Design 44

6.1.5 Data Flow and Interaction 49

6.1.6 Deployment View 50

6.2 Design Considerations 52

6.2.1 Database Design 52

6.2.2 User Interface Design 53

6.2.3 Security Design 54

6.2.4 Class Design 54

6.2.5 REST API Design 56

6.2.6 Key Algorithms 56

6.2.7 Design Decisions and Alternatives 59

6.3 Conclusion 59

A First Appendix if Required **67**

A.1 References 67

A.2 Equations 67
A.3 Figures 67
A.4 Tables 68
A.5 Pseudo Code 68
A.6 Code of Programming Languages 68
A.7 Recommendations for LaTeX 69

List of Figures

4.1 VibePen use case model 32
4.2 VibePen dashboard and scan history 34
4.3 New scan screen for code and verified URL targets 34
4.4 Domain ownership verification workflow 35
4.5 Scan progress and pipeline stages 35
4.6 Report view with findings and export action 36
4.7 Administration view with accounts and audit events 37
4.8 Proposed VibePen entity-relationship model 37
4.9 Qualitative likelihood and impact matrix for project risks 41

6.1 High-level architecture of VibePen 45
6.2 Ownership verification flow 47
6.3 Data flow of a VibePen scan 50
6.4 Sequence diagram of a scan 51
6.5 Architecture and class diagram of VibePen 52
6.6 Class diagram of the scanning service 55

A.1 Fast Logo is the Caption. 67

List of Tables

2.1 Objectives and proposed success measures 8
2.2 Constraints and their consequences 10
2.3 Stakeholders 12

3.1 Summary of related applications 19

4.1 UC-01: Authenticate User 26
4.2 UC-02: Submit Code Archive 27
4.3 UC-03: Verify Domain Ownership 28
4.4 UC-04: Run a Verified Website Scan 29
4.5 UC-05: Review and Export Report 30
4.6 UC-06: Administer Users and Rules 31
4.7 Data dictionary for the user entity 38
4.8 Data dictionary for the verified domain entity 38
4.9 Data dictionary for the scan entity 38
4.10 Data dictionary for the source artifact entity 39
4.11 Data dictionary for the finding entity 39
4.12 Data dictionary for the remediation entity 39
4.13 Data dictionary for the audit log entity 39
4.14 Project risks and mitigation measures 40

6.1 Proposed technology stack 45
6.2 Severity classification rules 49
6.3 Key database design considerations 53
6.4 Planned user-interface screens 53
6.5 Security threats and design responses 54
6.6 Main REST API endpoints 56
6.7 Key design decisions and alternatives 59

A.1 Give a Caption to the Table. 68

Chapter 1 Introduction

Vibe coding has made software development significantly more accessible by allowing users to describe an application in natural language and obtain a working implementation with limited traditional programming knowledge. AI-assisted development platforms such as Lovable, Bolt, Replit, and Base44 can generate complete web applications within a short period of time. This development approach reduces the technical barrier to software creation and enables individuals, students, freelancers, and small businesses to build applications without extensive programming experience. However, the speed and convenience of AI-generated development can also introduce security risks because generated applications may be deployed without undergoing a professional security review. Vulnerabilities may exist in authentication, authorization, database configuration, source code, dependencies, and application interfaces. These weaknesses can remain unnoticed when application creators do not have sufficient security knowledge. VibePen is proposed as an automated security scanner specifically designed for AI-generated web applications. The system combines static source-code analysis, controlled dynamic analysis, and an AI-assisted reasoning layer to identify, validate, prioritize, and explain security findings.

1.1 Purpose of this Document

The purpose of this document is to present the problem addressed by VibePen and describe the proposed solution, requirements, and design of the system. The document establishes the motivation for developing a security scanner specifically for applications created through AI-assisted development and explains the limitations of relying on conventional security tools for this emerging development approach. VibePen is intended to accept either a source-code archive or a verified live web application and analyze it for common security weaknesses. The document therefore provides the foundation for understanding how the proposed system will operate and why its different components are required. It describes the project vision, problem domain, scope, stakeholders, functional and non-functional requirements, and high-level and low-level design. The report also establishes the boundaries of the system so that development remains focused on vulnerability detection, evidence-based analysis, and understandable reporting rather than automatic code modification or unrestricted penetration testing. The document will serve as a reference throughout the implementation and evaluation phases of the Final Year Project.

1.2 Intended Audience

The primary audience of this report consists of the FYP supervisor and project evaluators who need to assess the relevance, feasibility, requirements, design, and expected outcomes of VibePen. The report is

also intended for the development team because it provides a common reference for implementing the system and understanding the reasoning behind its architectural and functional decisions. Security professionals and computer science students may use the report to understand how security analysis can be adapted to applications produced through AI-assisted development. Application owners and other non-expert users represent another important audience because VibePen is specifically designed to make security findings understandable to people who may not possess professional security expertise. The report is therefore written at multiple levels of abstraction. Technical details are provided where they are necessary to explain implementation decisions, while system objectives and security findings are described in accessible language where appropriate. By addressing these different audiences, the document provides both an academic description of the project and a practical reference for understanding the proposed system.

1.3 Definitions, Acronyms, and Abbreviations

The following definitions, acronyms, and abbreviations are used throughout this document to maintain consistency when describing the VibePen system.

- **Vibe Coding:** Building software by describing requirements in natural language and allowing an AI system to generate the implementation.
- **LLM:** Large Language Model used to process and generate natural language and code.
- **SAST:** Static Application Security Testing performed without executing the application.
- **DAST:** Dynamic Application Security Testing performed against a running application.
- **Semgrep:** A rule-based tool used to identify security-relevant code patterns.
- **Tree-sitter:** A parser used to construct syntax trees for source-code analysis.
- **Playwright:** A browser-automation framework used for controlled application crawling and interaction.
- **IDOR:** Insecure Direct Object Reference involving unauthorized access through manipulable object identifiers.
- **RLS:** Row-Level Security rules controlling access to individual database rows.
- **SSRF:** Server-Side Request Forgery in which a server is manipulated into making unintended requests.
- **Slopsquatting:** An attack involving registration of package names that AI systems commonly

hallucinate.

- **False Positive:** A reported security finding that does not represent an actual vulnerability.
- **OWASP:** Open Worldwide Application Security Project.
- **API:** Application Programming Interface.
- **REST:** Representational State Transfer.
- **JSON:** JavaScript Object Notation.
- **DNS TXT Record:** A DNS record containing text that can be used to verify domain control.
- **FR:** Functional Requirement.
- **NFR:** Non-Functional Requirement.
- **UC:** Use Case.

1.4 Conclusion

This chapter introduced VibePen and established the motivation for developing an automated security scanner for AI-generated web applications. It described how the increasing accessibility of AI-assisted development can create security challenges when generated applications are deployed without sufficient security review. The purpose and intended audience of the report were also established, providing the context needed to understand the remaining chapters. VibePen is intended to address these challenges by combining source-code analysis, controlled analysis of running applications, and AI-assisted reasoning to produce security findings that are both technically useful and understandable to non-expert application owners. The system is deliberately focused on detection and reporting rather than automatic modification of user applications. The next chapter develops the project vision in greater detail by defining the problem domain, problem statement, project objectives, scope, constraints, business opportunity, and stakeholder requirements. These elements establish the foundation from which the software requirements and system design will subsequently be developed.

Chapter 2 Project Vision

2.1 Problem Domain Overview

VibePen operates in the domain of web application security, with a specific focus on applications created through AI-assisted or "vibe coding" development. These applications are commonly generated by describing desired functionality in natural language and allowing an AI coding system to produce the required source code. The resulting applications may contain a front-end interface connected to a hosted database or Backend-as-a-Service platform such as Supabase or Firebase, with serverless functions providing additional application logic. Although this approach reduces development effort, the generated applications may contain security weaknesses because their creators may not have sufficient knowledge to identify insecure implementation choices. Vulnerabilities can occur in source code, authentication and authorization mechanisms, dependencies, database access rules, configuration, and interactions between different components. VibePen addresses this domain by providing automated security analysis for both uploaded source code and authorized live applications. Static analysis is used to identify source-level weaknesses, secrets, insecure patterns, and dependency issues, while dynamic analysis examines the behavior of running applications through controlled crawling and harmless test inputs. Findings from both approaches are normalized and passed to an AI-assisted reasoning layer that validates evidence, identifies relationships between findings, reduces false positives, and generates understandable explanations.

2.2 Problem Statement

Applications generated through AI-assisted development can be deployed without receiving an adequate security review. Application owners who rely on AI coding tools may have sufficient knowledge to describe the desired functionality of an application but may not possess the security expertise required to evaluate the generated implementation. As a result, vulnerabilities involving authentication, authorization, exposed credentials, database permissions, dependencies, input validation, and configuration may remain undetected. Existing security tools can identify many individual classes of vulnerabilities, but they are generally designed for professional developers and security specialists. Their output may contain technical terminology, rule identifiers, or other information that is difficult for non-expert users to interpret. General-purpose tools also do not necessarily focus on recurring security problems associated with AI-generated applications, such as hallucinated dependencies or permissive Backend-as-a-Service configurations. Furthermore, relying only on source-code analysis can miss runtime behavior, while relying only on dynamic analysis can miss weaknesses that are not reached during testing. There is

therefore a need for a system that combines multiple analysis approaches, focuses on common weaknesses of AI-generated applications, validates findings using evidence, and communicates the results in a form that application owners can understand and act upon.

2.3 Problem Elaboration

The problem addressed by VibePen consists of several related challenges. AI-generated applications may contain predictable security weaknesses that are difficult for non-experts to identify. At the same time, individual analysis techniques provide only a partial view of application security. Static analysis can inspect source code but cannot directly observe runtime behavior, whereas dynamic analysis can observe a running application but cannot reach every possible code path. Another challenge is the interpretation of security findings. A technically correct scanner result may still be of limited practical value if the user cannot understand its impact or determine how to address it. Finally, live security testing introduces authorization and safety concerns because scanning a website without permission may cause harm or violate applicable rules. VibePen therefore treats vulnerability detection, evidence validation, understandable reporting, and authorized testing as interconnected parts of the same problem rather than separate features.

2.3.1 Security Weaknesses Typical of AI-Generated Applications

AI-generated web applications may contain recurring security weaknesses because AI coding systems generally optimize for producing functional implementations rather than guaranteeing secure designs. VibePen focuses on weaknesses such as exposed credentials in source code or client-side bundles, missing or weak authentication, broken authorization including Insecure Direct Object References, injection vulnerabilities, permissive database and storage rules, insecure configuration, missing security headers, overly permissive Cross-Origin Resource Sharing settings, and unsafe or hallucinated dependencies. These weaknesses may occur independently or may interact with one another to create a more serious security issue. For example, an application can contain an authentication mechanism but still expose another user's information because authorization is not enforced on an individual data-access route. Similarly, credentials may be unintentionally included in client-side code even though the application appears to provide a secure login mechanism. The scanner must therefore examine more than the visible interface and should consider relationships between application components. This requirement motivates the use of multiple analysis techniques and an AI-assisted reasoning layer capable of considering evidence from different files and execution paths.

2.3.2 Limits of a Single Analysis Technique

Static analysis provides the ability to inspect source code, configuration files, dependencies, and other project artifacts without executing the application. This makes it useful for detecting exposed secrets, insecure coding patterns, suspicious dependencies, and configuration weaknesses. However, static analysis cannot directly determine how an application behaves during execution or whether a particular access-control rule is actually enforced at runtime. Dynamic analysis provides the opposite perspective by interacting with a running application and observing its responses, headers, network activity, and accessible functionality. Its limitation is that it can only inspect the application areas reached by the crawler and test cases. Some security weaknesses also require information from multiple sources. A route may appear protected in isolation while a related route or database policy bypasses the intended authorization mechanism. VibePen therefore combines static and dynamic analysis and uses an AI-assisted reasoning layer to correlate the resulting evidence. This approach is intended to provide broader coverage while allowing the system to distinguish between isolated warnings and findings that become significant when considered together.

2.3.3 Results That Non-Experts Cannot Use

Security scanners commonly produce technical findings containing rule identifiers, vulnerability classifications, stack traces, source-code locations, or other information intended for experienced developers and security professionals. Such information can be useful for diagnosing a vulnerability but may not be sufficient for an application owner who created the system through natural-language prompting. A non-expert user needs to understand what the vulnerability means, what an attacker could potentially achieve, how serious the issue is, and what should be changed to reduce the risk. VibePen therefore treats the presentation of findings as an important part of the security-analysis process. Findings are intended to be normalized, validated against available evidence, assigned an appropriate severity, and converted into explanations using accessible language. Where appropriate, the report can also provide practical remediation guidance that helps the user understand the type of change required. This approach does not replace the underlying technical evidence; instead, it translates the evidence into a form that can support decision-making by users with different levels of security expertise.

2.3.4 Authorized Testing

Live application security testing introduces an ethical and operational requirement that must be addressed by the system. A scanner that allows any user to submit any website could potentially be used to probe applications without the owner's permission. Such activity can expose sensitive infor-

mation, disrupt services, or create legal and ethical concerns. VibePen therefore restricts live scanning to applications for which the user has demonstrated ownership or authorization. The intended workflow begins with verification of domain control, after which the user can submit the authorized target for analysis. Dynamic checks are designed to use controlled and non-destructive requests rather than denial-of-service, brute-force, or data-extraction techniques. During development and evaluation, the project team will use applications that it owns or is explicitly entitled to test, including applications developed by the team, open-source applications executed in controlled environments, and intentionally vulnerable training applications. Authorization is consequently treated as a core system requirement rather than an optional security feature.

2.4 Goals and Objectives

The primary goal of VibePen is to develop and evaluate a working vulnerability-scanning system specifically tailored to AI-generated web applications. The system should be capable of accepting both source-code archives and authorized live applications, applying appropriate static and dynamic analysis techniques, and presenting the resulting findings through a unified reporting mechanism. The project also aims to reduce the difficulty of interpreting security findings for non-expert users without removing the technical evidence required by developers and security testers. An AI-assisted reasoning layer will be used to validate findings, connect evidence across files and components, and reduce false positives while remaining constrained by available evidence. The objectives listed in Table 2.1 define measurable targets that can be used during project evaluation. These targets represent initial values and may be refined in consultation with the project supervisor as the implementation and evaluation methodology becomes more detailed. The objectives also establish clear boundaries for the system so that project progress can be evaluated against measurable outcomes rather than only against the completion of individual software features.

2.5 Project Scope

The scope of VibePen is to design and develop an automated security scanner for web applications produced through AI-assisted development. The project focuses on analyzing source-code archives and authorized live applications, identifying selected classes of security weaknesses, validating findings using available evidence, and presenting the results through prioritized and understandable reports. The project does not attempt to provide a complete replacement for professional penetration testing because comprehensive security assessment requires expertise, broader testing methodologies, and potentially longer assessment periods. The scope is instead limited to a defined set of vulnerability classes and

Table 2.1: Objectives and proposed success measures

ID	Objective	Success Measure (Target)
O1	Dual-mode analysis of live URLs and uploaded source code.	Both modes produce one unified report format.
O2	Static analysis for secrets, insecure patterns, unsafe file structure, and non-existent dependencies.	At least 80% of seeded static vulnerabilities detected on the benchmark set.
O3	Dynamic analysis of verified live applications through a controlled crawler.	Crawler reaches at least 90% of pages and forms in test applications without sending destructive requests.
O4	LLM reasoning layer for multi-file flows and false-positive reduction.	At least 30% fewer false positives than raw tool output without losing true positives.
O5	Plain-language, severity-ranked reports with fix guidance.	Non-expert reviewers rate clarity at least 4 out of 5 in a usability check.
O6	Ownership verification before any live scan.	No live scan can start for an unverified domain, as verified through test cases.
O7	Evaluation on authorized benchmark applications.	Documented per-application results including seeded, detected, missed, and incorrect findings.

supported technologies that can be implemented and evaluated within the available project resources. The system is also designed around safe and authorized testing. Live application analysis cannot be performed against arbitrary websites, and dynamic analysis will avoid destructive activities. Defining the scope in this manner provides a clear boundary for development and evaluation while leaving potential extensions for future work. The following subsections describe the functionality that is included in the project and the functionality that is deliberately excluded.

2.5.1 In Scope

The following functionality is included within the scope of VibePen:

- Upload and offline analysis of JavaScript and TypeScript source-code archives.
- Analysis of Python back ends where project time permits.
- Analysis of live applications with verified domain ownership.
- Detection of exposed secrets and insecure coding patterns.
- Detection of authentication and authorization weaknesses.
- Detection of IDOR and selected injection vulnerabilities.
- Detection of insecure configuration and permissive access rules.
- Detection of hallucinated or potentially unsafe dependencies.

- LLM-assisted validation and cross-file finding correlation.
- Severity classification and prioritization of findings.
- Plain-language security reports with remediation guidance.
- Report export and scan-history functionality.

2.5.2 Out of Scope

The following functionality is excluded from the current project scope:

- Scanning websites without verified ownership or authorization.
- Automatic modification or patching of user source code.
- Security scanning of native mobile applications.
- Testing the security of AI coding platforms themselves.
- Extraction of private or sensitive data from target applications.
- Denial-of-service testing and brute-force attacks.
- Destructive exploitation beyond safe vulnerability verification.

The defined scope will be used as a guide throughout implementation and evaluation. Features that fall outside these boundaries may be considered as future extensions if sufficient time and resources remain after the core objectives have been completed.

2.6 Sustainable Development Goal (SDG)

VibePen primarily contributes to Sustainable Development Goal 9, Industry, Innovation and Infrastructure, which promotes resilient infrastructure, inclusive and sustainable industrialization, and innovation. The increasing use of AI-generated software means that applications created with these technologies are becoming part of the wider digital infrastructure used by individuals and organizations. Responsible adoption of this technology requires mechanisms through which its security can be evaluated, particularly when applications are created by users without professional software-security expertise. VibePen supports this objective by providing an automated mechanism for identifying selected security weaknesses in AI-generated web applications and presenting the results in a form that can be understood by non-expert users. By helping application owners identify security problems before they result in incidents, the project supports safer adoption of emerging software-development technologies. The contribution is therefore connected to both innovation and infrastructure resilience, as the project seeks

to make security assessment more accessible without requiring every application creator to become a security specialist.

2.7 Constraints

The development and evaluation of VibePen are subject to technical, ethical, operational, and resource constraints. These constraints influence the technologies that can be supported, the applications that can be tested, and the depth of vulnerability analysis that can be performed. The major constraints and their expected consequences are presented in Table 2.2. Authorization requirements limit live testing to verified or team-owned applications, which means that evaluation must primarily use controlled targets. The availability of suitable benchmark applications is another limitation because documented vulnerabilities specifically occurring in vibe-coded applications are relatively limited. The coverage of any automated scanner is also inherently incomplete because no collection of rules or crawler can identify every possible vulnerability. The use of an LLM introduces additional concerns because generated reasoning may be incorrect or unsupported by evidence. VibePen must therefore constrain AI-generated conclusions using the evidence produced by analysis tools. Finally, the project is limited by the available development time, team size, computational resources, and potential cost and latency of AI model calls. These constraints are considered when defining the project's architecture and evaluation criteria.

Table 2.2: Constraints and their consequences

Constraint	Description and Consequence
Authorization	Live testing is limited to verified or team-owned targets, so evaluation relies on controlled applications.
Test Data	Suitable benchmarks with documented vulnerabilities in vibe-coded applications are limited.
Tool Coverage	Detection is limited by the implemented security rules and crawler reach.
AI Reliability	LLM conclusions must remain tied to evidence and must not introduce unsupported findings.
Cost and Latency	LLM calls can introduce processing delays and financial costs.
Time	The academic-year schedule and three-member team limit technology and vulnerability coverage.
Safety of Scanning	Dynamic checks must remain non-destructive and rate-limited.

2.8 Business Opportunity

The increasing accessibility of AI-assisted development creates a growing group of application creators who can build and deploy software without possessing extensive security expertise. Individual creators, freelancers, students, start-ups, and small development teams may therefore have a practical

need for affordable security assessment tools that are easier to understand than professional penetration-testing platforms. VibePen addresses this opportunity by focusing specifically on applications produced through AI-assisted development and by presenting security findings in plain language. The system can potentially reduce the initial effort required to identify common weaknesses before an application is released or shared with users. Its dual analysis approach also provides an opportunity to combine source-code and runtime evidence within one report instead of requiring users to interpret multiple independent tools. Although the current project is an academic prototype and does not define a commercial deployment model, the concept could potentially be extended into a security-assurance service for AI-generated applications. Possible future extensions include continuous scanning, integration with development pipelines, automated security checks during deployment, and partnerships with AI-assisted development platforms.

2.9 Stakeholders Description/ User Characteristics

VibePen has several stakeholder groups whose requirements influence the design and functionality of the system. The primary stakeholder is the application owner or creator, who may have limited knowledge of application security and therefore requires clear explanations and practical remediation guidance. Developers are another important stakeholder group because they require technical evidence that allows vulnerabilities to be reproduced and corrected. Security testers require a controlled mechanism for performing authorized analysis, while administrators require mechanisms for managing users, monitoring activity, and preventing misuse. The project team is responsible for implementing and evaluating the system and therefore requires reproducible testing environments and measurable objectives. The supervisor and project evaluators require evidence that the project addresses a meaningful problem and satisfies its defined objectives. Third-party application owners are indirect stakeholders because their systems must be protected from unauthorized scanning. These different stakeholder characteristics require VibePen to balance usability, technical detail, security, authorization, and operational control.

2.9.1 Stakeholders Summary

The major stakeholders and their primary roles and requirements are summarized in Table 2.3.

2.9.2 Key High-Level Goals and Problems of Stakeholders

Application owners require a security assessment that explains vulnerabilities without assuming advanced security knowledge. Their primary concern is understanding whether an application is safe enough to deploy and what action should be taken when a problem is identified. Developers require more

Table 2.3: Stakeholders

Stakeholder	Role and Key Needs
Application Owner / Creator	Owns the application and may have limited security knowledge. Needs understandable findings, severity ranking, and practical fixes.
Developer	Maintains the application. Needs technical evidence such as affected files, lines, requests, and reproducible details.
Security Tester	Performs authorized testing. Needs controlled scanning and reliable evidence.
Administrator	Operates the platform. Needs user management, audit logs, access controls, and abuse prevention.
Project Team	Develops and evaluates VibePen. Needs reproducible applications and measurable evaluation results.
Supervisor / Evaluators	Assess scope, design quality, feasibility, implementation, and achievement of objectives.
Third-Party Site Owners	Must be protected from unauthorized or harmful scanning activity.

detailed technical evidence, including affected source-code locations and relevant requests or responses, so that findings can be reproduced and corrected. Security testers require reliable analysis capabilities while ensuring that testing remains authorized and non-destructive. Administrators require controls for managing users, maintaining audit records, and preventing misuse of the scanning functionality. The project team requires a system that can be tested against controlled applications and evaluated using measurable criteria. Supervisors and evaluators require evidence that the system addresses the stated problem and that its objectives have been achieved. Finally, third-party site owners require protection from unauthorized testing. These stakeholder needs influence the system's authorization mechanism, reporting design, analysis workflow, administrative controls, and safety restrictions.

2.10 Conclusion

This chapter established the vision and boundaries of VibePen as an automated security scanner for AI-generated web applications. The problem domain was described along with the security weaknesses that can occur when applications are produced rapidly through AI-assisted development without professional security review. The problem statement and its major sub-problems were then defined, including recurring security weaknesses, limitations of individual analysis techniques, difficulty interpreting technical findings, and the need for authorized testing. The project goals and measurable objectives were presented together with the functional boundaries defined through the project scope. The chapter also identified the project's contribution to Sustainable Development Goal 9 and discussed the technical, ethical, and resource constraints that may affect development. Finally, the major stakeholders and their requirements were identified to establish the user and operational perspective of the system. These deci-

sions provide the foundation for the software requirements and system design that will be described in the following chapters.

Chapter 3 Related Applications

3.1 Introduction

This chapter reviews existing applications and research related to the security of web applications, with particular attention to projects created quickly through prompts and AI coding assistants. The review covers static application security testing, source-code parsing, secret detection, dependency analysis, dynamic testing, browser automation, and intentionally vulnerable applications used for evaluation. These systems solve different parts of the problem. Some inspect source code, some check dependencies against known vulnerability databases, and others test a running application. Their results are useful, but users may need technical experience to configure the tools and understand their output. Research on code generated by AI assistants also shows why security review is important: generated code can be functional while still containing security weaknesses [1]. The chapter therefore compares existing systems by their purpose, inputs, strengths, and limitations rather than assuming that one tool can identify every vulnerability. Product capabilities described here are based on the cited documentation and research. Since software tools change over time, their latest documentation should be checked again before the final submission. The review first discusses the available applications and test environments, then identifies the remaining gap and explains how VibePen is positioned in relation to them.

3.2 Static Analysis and Source-Code Parsing

Static application security testing (SAST) examines source code without running the application. It can identify suspicious patterns early in development and is useful when an AI-generated project is available as a folder or repository. Semgrep applies configurable rules to source code and supports custom rules for project-specific checks [4]. CodeQL represents code as a queryable database, allowing queries to trace data flow and find certain vulnerability patterns across functions [7]. SonarQube combines code-quality checks with security analysis and can be integrated into development workflows [8]. These tools provide complementary approaches: rule-based matching can be fast and easy to customize, while query-based analysis can express more complex relationships. However, their results still require interpretation. A match does not always prove that a vulnerability can be exploited, and a clean scan does not prove that an application is secure. Coverage also depends on supported languages, rules, configuration, and the information available to the scanner. VibePen can use these tools as evidence sources, but it should not present their findings as complete proof of security or treat the absence of findings as a guarantee.

Source-code structure can help connect findings to the application in which they occur. Tree-sitter is a parser that builds syntax trees and is designed to work efficiently with source code, including code

that may be incomplete during editing [5]. It is not itself a vulnerability scanner; instead, its syntax information can help identify imports, function calls, routes, and other code structures. Bandit focuses on common security issues in Python code by examining its abstract syntax tree and applying security-related plugins [9]. These projects illustrate two different roles: a parser provides structural information, while a security linter checks for selected patterns. Combining them can help a review system connect a finding to relevant code, but the combination still has limits. Parsing does not establish whether a route is reachable, and a rule-based warning may lack runtime context. For VibePen, source parsing is therefore treated as supporting evidence that helps organize code and guide analysis, rather than as an independent decision about whether a complete application is safe.

3.3 Secret and Dependency Scanning

Secret-scanning tools search source code and repositories for values that may be credentials, tokens, or other sensitive information. Gitleaks detects potential secrets in repositories and files using configured patterns and related heuristics [10]. This is important for generated projects because developers may accidentally include example credentials or copy values into source files. However, a secret scanner addresses a narrower problem than application security as a whole. A detected string may require verification, while credentials stored outside the scanned files may not be found. Dependency-scanning tools address a different risk: they identify libraries used by a project and compare them with records of known vulnerabilities. Snyk provides dependency and code-security capabilities documented for development workflows [11]. OWASP Dependency-Check identifies project dependencies and checks for publicly disclosed vulnerabilities associated with them [12]. Trivy can scan software components and other supported targets for known vulnerabilities [13]. These tools are useful for finding known issues in declared or detected components, but their coverage depends on accurate package identification and current vulnerability data. They do not automatically prove that a dependency is safe, nor do they necessarily establish whether a vulnerable component is reachable in the application.

AI-generated projects also create a related package-integrity concern: a generated project may refer to a package name that is incorrect or does not correspond to the intended real package. Ordinary vulnerability scanning is not sufficient for every such case because a scanner may only assess dependencies it can identify. A separate verification step can compare dependency declarations with the relevant package registry and project metadata. This check should report uncertainty when package identity cannot be established, rather than labelling every unfamiliar name as malicious. The same cautious approach applies to secrets and dependency findings: each result should include the file, package, or evidence that caused it to be reported. The OWASP Top 10 describes broad categories of web application risk, including risks

related to vulnerable and outdated components, while the Common Weakness Enumeration provides a shared classification of software weaknesses [2, 3]. These references can help categorize findings consistently. They are classification and guidance resources, however, not replacements for scanning or testing an actual application.

3.4 Dynamic Analysis and Browser Automation

Dynamic application security testing (DAST) tests a running application by sending requests and observing responses. OWASP ZAP provides an open-source proxy and scanning tools for web application testing [14]. Burp Suite offers a toolkit used for manual and automated web security testing [15]. Nuclei checks targets against templates for known vulnerabilities and configuration issues [16]. These applications can identify issues that are not obvious from source code alone, such as certain runtime behaviours or server responses. Their effectiveness depends on what they can reach and test. Authentication, application state, rate limits, and crawling coverage may affect results, while active tests can create unwanted effects if they are run against systems without permission. The tools also produce findings that may need technical interpretation. A safe workflow should limit scans to authorized targets, prefer non-destructive checks where possible, and record enough evidence for a user to review a result. A scanner should not be run against a public site merely because its URL is available; permission and scope must be established first.

Modern web applications may render much of their interface in the browser, so a simple HTTP crawler may not discover every page or action. Playwright automates browsers and can interact with web pages in a way similar to a user, making it useful for navigation and repeatable functional checks [6]. Browser automation can help a testing system discover reachable routes and collect responses, but it is not automatically a security scanner. Its coverage depends on the actions, test accounts, and application states configured for a run. Dynamic analysis also benefits from an understanding of common security risks. The OWASP Server-Side Request Forgery Prevention Cheat Sheet, for example, provides guidance on reducing SSRF risks, which can inform checks and interpretation when relevant to the application architecture [17]. Such guidance should be translated into bounded, safe tests rather than indiscriminately probing arbitrary systems. Together, proxy-based scanners, template scanners, and browser automation offer different forms of runtime evidence. A combined workflow can make use of each capability, provided that the system preserves the distinction between a detected pattern, a confirmed behaviour, and a vulnerability that has been validated in context.

3.5 AI-Assisted Security Analysis

Large language models can help explain technical findings, summarize relevant source code, and suggest possible remediation steps. This may be helpful for users who do not have professional security experience. However, a model-generated explanation is not itself proof that a vulnerability exists. Models can misunderstand code, miss relevant context, or produce confident but incorrect statements. Research examining code generated by AI assistants has reported security weaknesses in generated snippets and has explored whether static-analysis feedback can help address some of those issues [1]. This supports the need for independent checks rather than relying only on an AI assistant's own assessment. Another concern is prompt injection: source files, comments, web pages, or other content being analyzed may contain instructions intended to influence the model. Such content must be treated as untrusted data and must not override the system's analysis instructions. A safer design gives the model bounded tasks, supplies findings from deterministic tools, and asks it to explain the available evidence. The model should be able to mark a result as uncertain or unsupported. This approach can improve readability while reducing the risk that a fluent explanation is mistaken for a verified security conclusion.

3.6 Evaluation Applications and Test Environments

A security system needs test applications whose relevant weaknesses are known or can be established independently. OWASP Juice Shop is an intentionally insecure web application designed for security training and demonstrations [18]. Damn Vulnerable Web Application (DVWA) provides a deliberately vulnerable environment for practicing common web security tests [19]. OWASP WebGoat is another educational application designed to teach web application security through lessons and exercises [20]. These projects are useful for controlled evaluation because they provide known examples of vulnerable behaviour. They should be deployed in a local or otherwise authorized environment and should not be treated as ordinary production sites. A separate test set can contain open-source projects that are reviewed offline for insecure patterns, exposed secrets, and vulnerable dependencies. The team can also build small applications from realistic prompts using AI coding tools, then inspect them and record the weaknesses that are deliberately included or independently confirmed. This provides examples closer to the target setting, although the generated applications must be documented carefully so that the evaluation is reproducible. Test results should distinguish between vulnerabilities known to be present, findings reported by the system, missed issues, and incorrect findings.

For each application, the evaluation should record the version or commit, configuration, test conditions, and the evidence used to establish the expected results. The same scope and test procedure should

be applied when comparing VibePen with individual tools, wherever their capabilities overlap. Suitable measures include recall, precision, and false-positive rate; these values should be calculated from recorded outcomes rather than estimated or invented. The test set should include more than one vulnerability category and more than one application so that results do not depend on a single example. If an application requires authentication, the test account and accessible features should be recorded without exposing real credentials. The PostgreSQL documentation on JSON types is relevant only if a component of the proposed system stores or queries structured JSON data in PostgreSQL; it does not provide evidence about vulnerability detection itself [21]. More generally, documentation and benchmark descriptions should be used only for claims they directly support. These evaluation practices help separate the capabilities of the existing applications from the proposed system's own measured performance.

3.7 Comparison of Related Applications

The reviewed applications cover different parts of a security assessment, and their outputs are not directly interchangeable. Table 3.1 compares representative tools by their primary role and the limitation that remains when they are used for AI-generated web applications. The table is a summary rather than a claim that every product has been tested under identical conditions. The descriptions are based on the cited documentation; the practical effectiveness of each tool depends on its version, configuration, supported languages, and the application being assessed. The comparison also distinguishes parsers and test environments from scanners, since a parser or intentionally vulnerable application serves a different purpose from a product that reports security findings. VibePen is included as the proposed system for comparison, but its capabilities should be evaluated experimentally rather than assumed from the design. In particular, a unified interface does not by itself establish better detection accuracy. That claim would require a reproducible evaluation using known test cases and recorded outcomes.

3.8 Identified Gap

The comparison indicates that existing applications provide useful but different capabilities. Static analysis examines source code, secret scanners focus on sensitive values, dependency scanners check known component risks, and dynamic scanners test reachable behaviour. Parsers and browser automation can support these tasks, while intentionally vulnerable applications help evaluate them. These functions are often used through separate tools, and their outputs can vary in format and level of detail. The literature on AI-generated code provides an additional reason to review generated applications rather than assume that code which runs correctly is also secure [1]. Based on the reviewed sources, the design opportunity for VibePen is to combine evidence from these established approaches in one guided workflow, include

Table 3.1: Summary of related applications

Tool	Primary role	Main input	Main strength	Limitation in the VibePen context
Semgrep	Static analysis	Source code	Configurable rules and custom checks	Findings depend on available rules and may need contextual validation.
CodeQL	Static analysis	Source and database	Data-flow queries across code	Database setup and query knowledge may be required.
SonarQube	Code quality and security	Source code	Reporting and development-workflow integration	Results may require technical interpretation and do not specifically cover every AI-generation risk.
Tree-sitter	Source parsing	Source code	Structured syntax trees	Parsing alone does not determine whether code is vulnerable.
Bandit	Python security linting	Python source	Checks selected Python security patterns	Focused on Python and the rules it supports.
Gitleaks	Secret detection	Files and repositories	Identifies possible exposed secrets	Does not cover general application vulnerabilities.
Snyk	Dependency and code security	Code and manifests	Checks supported projects for known risks	Coverage depends on supported ecosystems and available vulnerability data.
Dependency-Check	Dependency analysis	Project dependencies	Maps identified components to known vulnerability records	Component identification may be uncertain or incomplete.
Trivy	Vulnerability scanning	Supported project and system targets	Checks supported components against vulnerability data	Results depend on target support and component identification.
OWASP ZAP	Dynamic testing	Running web app	Open-source proxy and scanner	Crawl coverage and safe configuration affect findings.
Burp Suite	Web security testing	Running web app	Manual and automated testing toolkit	Some workflows require security-testing experience.
Nuclei	Template-based scanning	Authorized target	Checks known patterns at scale	Template coverage does not provide complete application reasoning.
Playwright	Browser automation	Web application	Automates browser interactions	Requires configured actions and is not a security scanner by itself.
Juice Shop, DVWA, WebGoat	Evaluation environments	Test deployment	Provide deliberately vulnerable examples	Do not represent every real-world or AI-generated application.
VibePen	Proposed combined workflow	Code or authorized URL	Combines analysis evidence with plain-language explanations	Must be validated for coverage, accuracy, supported targets, and safe operation.

checks relevant to prompt-driven development, and present results in language that non-expert users can understand. This is a proposed design goal, not a claim that the system already detects more vulnerabilities than existing tools. Its effectiveness must be established through testing against documented examples and suitable baselines.

3.9 Positioning of VibePen

VibePen is intended to sit between professional security tooling and people who create applications with AI coding assistants. The proposed workflow uses established scanners and source-analysis methods where they are suitable, then organizes the resulting evidence into findings that can be reviewed. Each finding should identify the relevant file, code location, dependency, request, or response when that evidence is available. The AI component should explain evidence and possible remediation rather than invent a vulnerability when the underlying tool output does not support it. The system should also distinguish confirmed findings from possible issues and indicate when a check could not be completed. These design choices are intended to make results more understandable without hiding uncertainty. VibePen is not intended to replace professional penetration testing, guarantee that an application is secure, or bypass the need for human review in high-impact situations. Its contribution will depend on the breadth of supported projects, the quality of its checks, the reliability of its explanations, and the results of its evaluation.

3.10 Ethical and Legal Considerations

Web security scanners can be configured to send requests to live services, so the system must establish authorization and scope before dynamic testing. A public URL is not, by itself, permission to scan a website. VibePen should require the user to confirm that they own the application or have explicit permission to test it, and it should limit scans to the approved host and scope. Rate limits, timeouts, non-destructive checks, and audit logs can reduce operational risk and make test activity traceable. The system should avoid actions that could modify data, disrupt service, or access information beyond the agreed scope. Where authorization cannot be established, the live scan should not proceed; code can instead be analyzed offline if the user is entitled to provide it. These controls are particularly important for a tool designed for non-expert users, who may not understand the effects of active scanning. The system should also handle submitted source code, secrets, and reports carefully, avoid exposing credentials in generated output, and explain the limits of its findings. Ethical safeguards are part of the system requirements rather than optional additions to the scanner.

3.11 Conclusion

This chapter reviewed static analysis, source parsing, secret detection, dependency scanning, dynamic testing, browser automation, AI-assisted review, and vulnerable applications used for evaluation. The reviewed projects provide established techniques for finding different kinds of issues, but no single tool guarantees complete security coverage. Research on AI-generated code further supports the need to test generated applications rather than assume they are secure because they execute successfully. VibePen is proposed as a guided workflow that combines evidence from existing tools, adds checks relevant to prompt-driven applications, and explains results in accessible language. Its value must be demonstrated through controlled evaluation, transparent reporting of false positives and missed vulnerabilities, and safe authorization controls for live testing. The next stage of the project should therefore implement the planned workflow and evaluate it on documented test applications using repeatable procedures and measured results. The report should include the test configuration and limitations so that readers can understand and reproduce the evaluation.

Chapter 4 Software Requirement Specifications

This chapter specifies the functional behavior, quality constraints, assumptions, user interactions, data model, and principal risks of VibePen. VibePen is intended to help developers inspect code archives and eligible websites for security weaknesses that may occur in rapidly assembled or AI-assisted applications. The requirements distinguish deterministic evidence collection from the language-model-assisted explanation of findings. The system must preserve traceability from a reported issue to the evidence and remediation advice that support it. Because scanning can affect systems outside the project, website scans are restricted to domains for which the user has completed ownership verification. The requirements in this chapter are the basis for the architecture, implementation, and evaluation described later in the report.

4.1 List of Features

VibePen provides account-based access, code-archive scanning, verified-domain management, controlled website scanning, scan progress tracking, evidence-backed findings, remediation guidance, report export, and administrative audit facilities. The code workflow accepts an archive for inspection without executing its contents. The website workflow requires a current ownership proof before a scan can begin. Static rules and other deterministic tools collect candidate findings; a reasoning layer may explain or prioritize those findings but must not create unsupported vulnerabilities. Users can review severity, location, redacted evidence, status, and recommended corrective steps. Administrators can manage users and rule configurations and inspect audit events. These features are intended to keep the process understandable and reproducible while limiting unnecessary collection of sensitive information. The feature set is a project specification and should be revised if implementation decisions change.

4.2 Functional Requirements

The following requirements describe observable system behavior. FR identifiers are retained so that implementation and test cases can trace back to the specification.

FR-01 The system shall allow a user to create an account using a unique email address and a securely hashed password.

FR-02 The system shall authenticate users and enforce role-based access for developers and administrators.

FR-03 The system shall allow an authenticated developer to upload a supported source-code archive.

FR-04 The system shall validate archive type, size, and structure before analysis.

FR-05 The system shall reject unsafe archives and extract accepted files in an isolated, non-executing workflow.

FR-06 The system shall calculate and retain a checksum for each accepted source artifact.

FR-07 The system shall enforce upload limits and reject path traversal and archive-expansion attacks.

FR-08 The system shall let a developer register a domain for ownership verification.

FR-09 The system shall support DNS TXT, verification-file, or HTML meta-tag proof methods.

FR-10 The system shall record verification time and expiry and require re-verification after expiry.

FR-11 The system shall prevent URL scans unless the target is covered by a valid ownership proof.

FR-12 The system shall block private, loopback, link-local, and otherwise disallowed network destinations and re-check redirect targets.

FR-13 The system shall create a scan record with a unique identifier, owner, mode, and status.

FR-14 The system shall expose scan states including queued, running, completed, and failed.

FR-15 The system shall run configured static checks against accepted source files.

FR-16 The system shall support dependency and exposed-secret checks where the relevant inputs are available.

FR-17 The system shall perform only configured, non-destructive dynamic checks on verified websites.

FR-18 The system shall enforce request-rate limits and stop dynamic checks when a safety threshold is reached.

FR-19 The system shall normalize tool output into a common finding format.

FR-20 Each finding shall include a category, severity, title, location, evidence, source engine, and status.

FR-21 The system shall redact detected secrets from stored evidence and from context sent to an external reasoning service.

FR-22 The reasoning layer shall receive only the minimum relevant evidence needed to explain a candidate finding.

FR-23 The reasoning layer shall return schema-constrained explanations and remediation suggestions.

FR-24 The system shall reject malformed reasoning output and shall not treat unsupported model claims as confirmed findings.

FR-25 The system shall associate each finding with remediation guidance and, where available, a public reference.

FR-26 The system shall calculate and display an overall risk score using a documented scoring method.

FR-27 The system shall display scan progress and report completion or failure.

FR-28 The system shall let a developer view scan history and open a report.

FR-29 The system shall let a developer mark a finding as open, false positive, accepted, or fixed.

FR-30 The system shall support exporting a report containing findings, evidence summaries, and remediation guidance.

FR-31 The system shall disclose when external services may process redacted scan context.

FR-32 The system shall retain provenance linking each finding to its source tool and supporting evidence.

FR-33 The system shall label uncertain or unvalidated findings for human review.

FR-34 The system shall allow administrators to manage user access.

FR-35 The system shall allow authorized administrators to manage approved scanning rules and tool versions.

FR-36 The system shall record security-relevant actions in an audit log.

FR-37 The system shall allow administrators to inspect audit events and relevant scan metadata.

FR-38 The system shall enforce authorization checks for every scan, finding, artifact, and report request.

FR-39 The system shall support deletion or expiry of retained upload artifacts according to the retention policy.

FR-40 The system shall record the versions of scanning tools used to produce a report.

FR-41 The system shall record blocked scan attempts and administrative changes in the audit log.

FR-42 The system shall provide actionable error messages without exposing secrets, internal paths, or stack traces.

4.3 Quality Attributes and Non-Functional Requirements

The quality attributes guide decisions that cannot be expressed only as features. Security and privacy are primary because both uploaded code and remote pages are untrusted. Reliability requires that partial tool failures be visible rather than silently presented as a complete scan. Usability requires that developers can move from a finding to its location, evidence, and remediation without needing to interpret raw

scanner output. Maintainability requires modular adapters and recorded tool versions so that a rule update can be traced to a changed result. The following measurable targets are proposed for evaluation; they should be confirmed against the available development hardware and final deployment environment.

NFR-01 Passwords and verification tokens shall be stored only as cryptographic hashes.

NFR-02 All protected application endpoints shall require authentication and object-level authorization.

NFR-03 Accepted uploads shall be analyzed without executing their contents.

NFR-04 Uploads shall be stored in an access-restricted encrypted volume and deleted according to a documented retention period.

NFR-05 Stored evidence and model-bound context shall be redacted for detected secrets.

NFR-06 The interface shall identify scan state and distinguish confirmed tool evidence from model-generated explanation.

NFR-07 Dynamic checks shall be non-destructive, rate-limited, and interruptible.

NFR-08 For a defined evaluation corpus, the system shall report precision, recall, and false-positive rate rather than relying only on an aggregate score.

NFR-09 For a typical small project archive on the target machine, the team shall measure end-to-end scan duration and report the observed median and 95th percentile.

NFR-10 The interface shall remain usable at a desktop viewport of at least 1280 by 720 pixels.

NFR-11 The system shall use modular scanner adapters so that a tool can be replaced without rewriting report presentation.

NFR-12 Each report shall record scanner and rule versions for reproducibility.

NFR-13 The privacy notice shall explain what data is stored and what redacted context may be sent to an external service.

NFR-14 Security-relevant events shall be recorded in an append-oriented audit log with timestamps.

4.4 Assumptions and Constraints

The initial implementation assumes that users can provide supported JavaScript or TypeScript projects as archives and can access the verification mechanism for any website they request to scan. Ownership verification is evidence of control for the configured domain and is not a guarantee that every path or third-party service is safe to test. The scanner must therefore apply scope checks and conservative request limits even after verification. Availability and cost of an external language-model service may

vary; the system must preserve deterministic tool findings if the reasoning layer is unavailable. The evaluation dataset may be limited, so results must state the tested languages, vulnerability categories, and corpus composition. The design also assumes that administrators can maintain approved rule versions and retention settings. These assumptions are planning constraints, not claims that the final prototype has already satisfied every target.

4.5 Use Cases

The primary actors are the developer, administrator, scanning tools, and optional external reasoning service. The developer uploads code or requests a scan of a verified domain, follows progress, and reviews a report. The administrator manages accounts and approved rules and reviews audit events. Scanner adapters return structured candidate findings; the reasoning service, if enabled, explains only the evidence it receives. The use cases below describe central workflows with security or architectural significance. The actual behavior of the completed prototype should be checked against these descriptions during testing.

4.5.1 UC-01: Authenticate User

Table 4.1: UC-01: Authenticate User

Name	Authenticate User
Actors	Developer, Administrator
Summary	Allows a developer or administrator to sign in and access their authorized workspace.
Pre-Conditions	The account exists and is enabled.
Post-Conditions	A valid user has an authenticated session; failed authentication is recorded according to the audit policy.
Special Requirements	Authentication failure messages must not reveal whether a particular email address is registered.
Basic Flow	
Actor Action	**System Response**
1. The user opens the sign-in page and enters credentials.	2. The system accepts the submitted credentials for validation.
3. The user submits the credentials.	4. The system validates the credentials without revealing whether a particular email is registered.
5. The user provides valid credentials.	6. The system establishes an authenticated session and loads the user's authorized workspace.
Alternative Flow	
If credentials are invalid or the account is disabled, the system displays a generic failure message and does not create a session.	

4.5.2 UC-02: Submit Code Archive

Table 4.2: UC-02: Submit Code Archive

Name	Submit Code Archive
Actors	Developer, archive validator, scanner worker
Summary	Allows an authenticated developer to submit a supported code archive for static analysis.
Pre-Conditions	The developer is authenticated and the archive is within configured upload limits.
Post-Conditions	A valid archive is queued for analysis, or an invalid archive is rejected without execution.
Special Requirements	Archive validation must check file type, size, checksum, structure, unsafe paths, and expansion patterns. Accepted files must be extracted in isolation.
Basic Flow	
Actor Action	**System Response**
1. The developer selects a supported archive and submits it for scanning.	2. The system checks the file type, size, checksum, and archive structure.
3. The developer submits an archive that passes validation.	4. The system rejects unsafe paths and expansion patterns and extracts accepted files in isolation.
5. The developer waits for the scan to be initialized.	6. The system creates a scan record, queues configured static checks, and displays the scan identifier and current state.
Alternative Flow	
If validation fails, the system rejects the archive with a safe explanation and records the blocked attempt.	

4.5.3 UC-03: Verify Domain Ownership

Table 4.3: UC-03: Verify Domain Ownership

Name	Verify Domain Ownership
Actors	Developer, verification service
Summary	Enables a developer to prove control of a domain before requesting scans of its website.
Pre-Conditions	The developer is authenticated and supplies a syntactically valid domain.
Post-Conditions	The domain is eligible for scoped URL scans only while verification remains valid.
Special Requirements	Verification tokens must be randomly generated, and the system must store the token hash rather than the raw token. Verification expiry must be recorded.
Basic Flow	
Actor Action	**System Response**
1. The developer registers the domain and chooses DNS, file, or meta-tag verification.	2. The system issues a random verification token and stores its hash.
3. The developer publishes the proof using the selected method.	4. The system checks the proof and records the verification time and expiry.
Alternative Flow	
Missing, incorrect, or expired proof leaves the domain unverified. URL scanning remains unavailable until verification succeeds.	

4.5.4 UC-04: Run a Verified Website Scan

Table 4.4: UC-04: Run a Verified Website Scan

Name	Run a Verified Website Scan
Actors	Developer, URL scanner, optional reasoning service
Summary	Allows a developer to run configured non-destructive security checks against a target covered by a verified domain and review the resulting report.
Pre-Conditions	The user is authenticated and the target is covered by an unexpired verified domain.
Post-Conditions	The user can inspect a report, including any incomplete stages or review warnings.
Special Requirements	Scanning must enforce target scope and rate limits. Candidate findings must be normalized, redacted, and stored with provenance. Any optional reasoning service must receive minimal redacted evidence and return a schema-validated explanation.
Basic Flow	
Actor Action	**System Response**
1. The developer selects the verified domain and starts a scan.	2. The system validates the target, scope, DNS resolution, and redirect destinations.
3. The developer requests the scan to proceed.	4. The scanner runs configured non-destructive checks within rate limits.
5. The scan tools return candidate findings.	6. The system normalizes, redacts, and stores findings with provenance.
7. The developer requests or uses the report when processing finishes.	8. If enabled, the reasoning service receives minimal redacted evidence and returns a schema-validated explanation. The system assembles the report and marks the scan complete.
Alternative Flow	
A scope violation, unsafe destination, rate-limit breach, or tool failure blocks or stops the scan and records the outcome.	

4.5.5 UC-05: Review and Export Report

Table 4.5: UC-05: Review and Export Report

Name	Review and Export Report
Actors	Developer
Summary	Allows an authorized developer to inspect scan results, review findings, update finding status, and export a report.
Pre-Conditions	The developer owns or is authorized to access the scan.
Post-Conditions	The report is available to authorized users and status changes are auditable.
Special Requirements	Access to reports and scan findings must be restricted to authorized users. Incomplete scan stages must be clearly identified.
Basic Flow	
Actor Action	**System Response**
1. The developer opens a completed scan from history.	2. The system displays the risk summary and findings by severity.
3. The developer opens a finding.	4. The system displays its location, redacted evidence, explanation, and remediation steps.
5. The developer updates the finding status and exports the report if required.	6. The system records the status change and provides the report export when requested.
Alternative Flow	
If the scan failed or is incomplete, the system displays available results and clearly marks missing stages.	

4.5.6 UC-06: Administer Users and Rules

Table 4.6: UC-06: Administer Users and Rules

Name	Administer Users and Rules
Actors	Administrator
Summary	Allows an administrator to manage user accounts and approved settings and review permitted audit events.
Pre-Conditions	The user has an administrator role.
Post-Conditions	Approved changes take effect and the audit record is available for review.
Special Requirements	Administrative changes must be validated and recorded with the actor, action, target, and timestamp. Access to audit events must follow policy.
Basic Flow	
Actor Action	**System Response**
1. The administrator opens the administration view.	2. The system displays user accounts, approved rule settings, tool versions, and audit events permitted by policy.
3. The administrator changes an authorized setting or account state.	4. The system validates the change and records the actor, action, target, and timestamp.
Alternative Flow	
Unauthorized or invalid changes are rejected and logged where appropriate.	

4.6 Use Case Model

The use case model in Figure 4.1 summarizes the principal interactions between the developer or application owner, the administrator, VibePen, the target web application, and the external language-model service. It includes account access, source-code submission, live URL submission, ownership verification, scan configuration, progress monitoring, report review and export, scan history, finding triage, and administrative audit review. The model describes the intended system boundary and should be kept consistent with the implemented workflows.

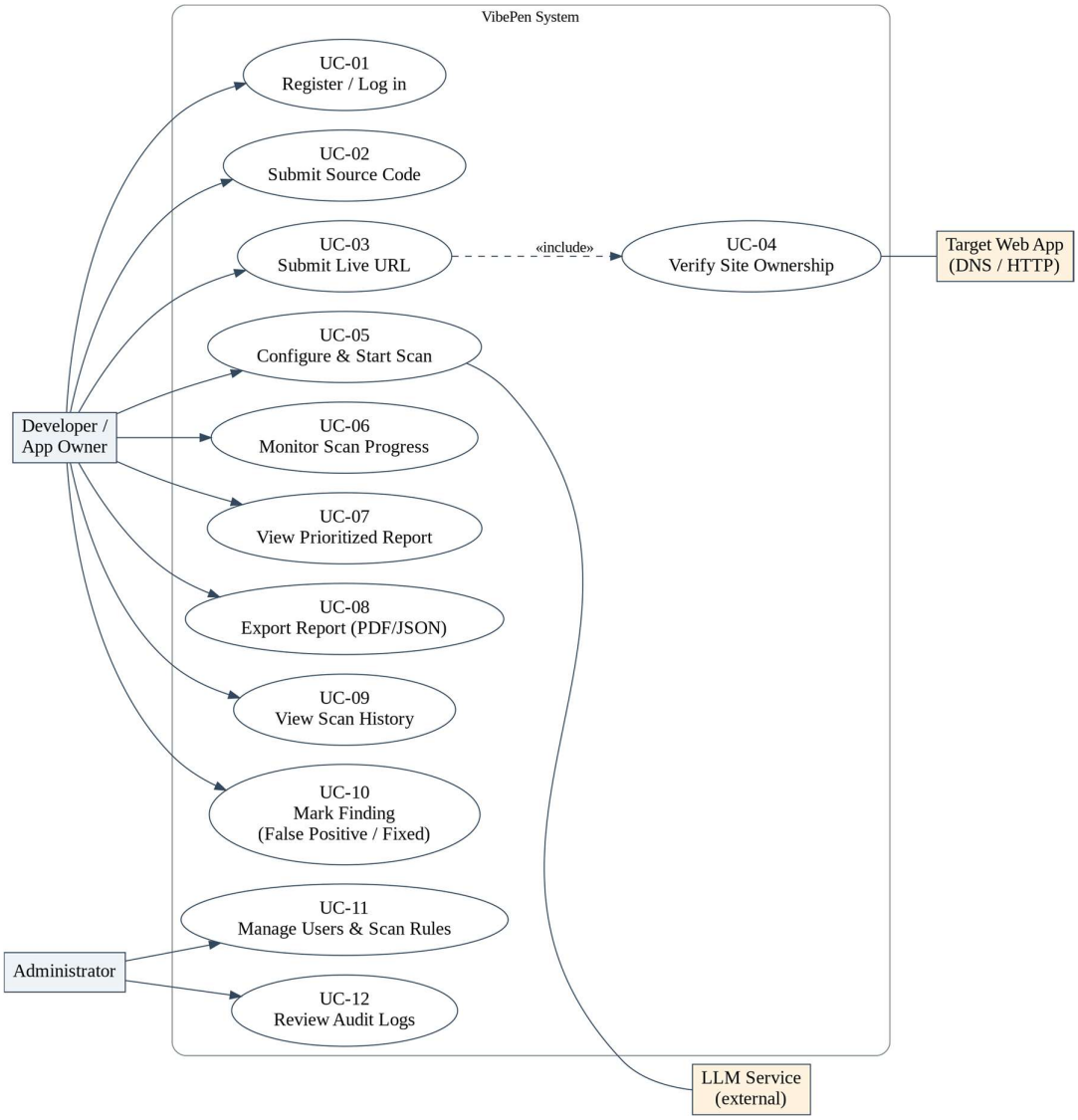


Figure 4.1: VibePen Use Case model

4.7 Hardware and Software Requirements

The development environment requires a modern multi-core computer, sufficient memory for archive extraction and scanner processes, and network access for package installation and optional model-service calls. A practical prototype target is a four-core CPU, 8 GB RAM, and at least 10 GB of free storage, with additional space depending on the size and retention of test archives. These figures are planning estimates and should be replaced with measured deployment requirements after performance testing. The software stack should include a supported operating system, Python or the selected backend runtime, a web frontend, PostgreSQL, approved static-analysis tools, a container or sandbox facility for isolated

processing, and version control. Exact versions must be pinned in the implementation and included in the final deployment documentation.

4.7.1 Hardware requirements

- Development machine: four or more CPU cores and at least 8 GB RAM.
- Storage: at least 10 GB free for source, database, logs, and test artifacts.
- Network: internet access for dependency installation and optional external reasoning calls.

4.7.2 Software requirements

- Operating system: a currently supported Linux, Windows, or macOS development environment.
- Database: PostgreSQL with JSONB support for structured, variable-shaped evidence.
- Backend and frontend runtimes: versions pinned in the project configuration.
- Security tools: approved static rules, dependency inspection, and optional safe HTTP checks.
- Isolation: restricted worker/container execution and controlled artifact storage.

4.8 Graphical User Interface

The interface mockups define the intended information architecture rather than claiming that every screen is already implemented. The dashboard summarizes scan activity and gives developers access to recent results. The new-scan screen separates code archives from URL targets so that the relevant validation rules are clear before submission. The ownership wizard explains how a user proves control of a domain and prevents the URL workflow from appearing available before verification. The progress view communicates each pipeline stage, while the report view presents severity, location, evidence, and remediation. The administration view separates privileged user and rule-management actions from developer workflows. The figures are illustrative design assets and should be updated if the final interface differs.

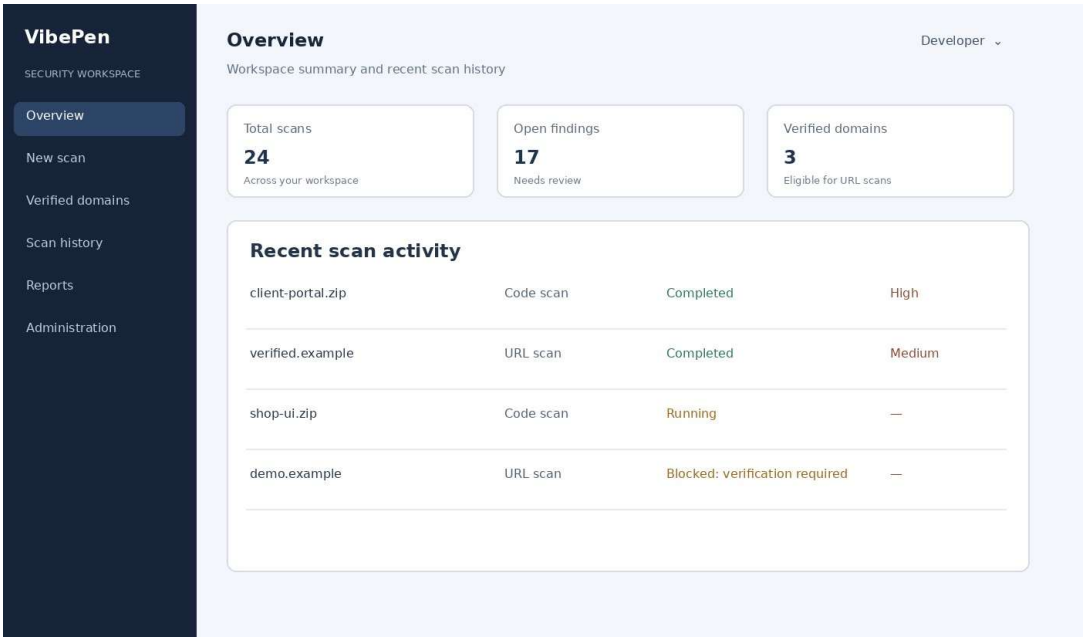


Figure 4.2: VibePen dashboard and scan history

The dashboard in Figure 4.2 is the developer's entry point for monitoring scan activity.

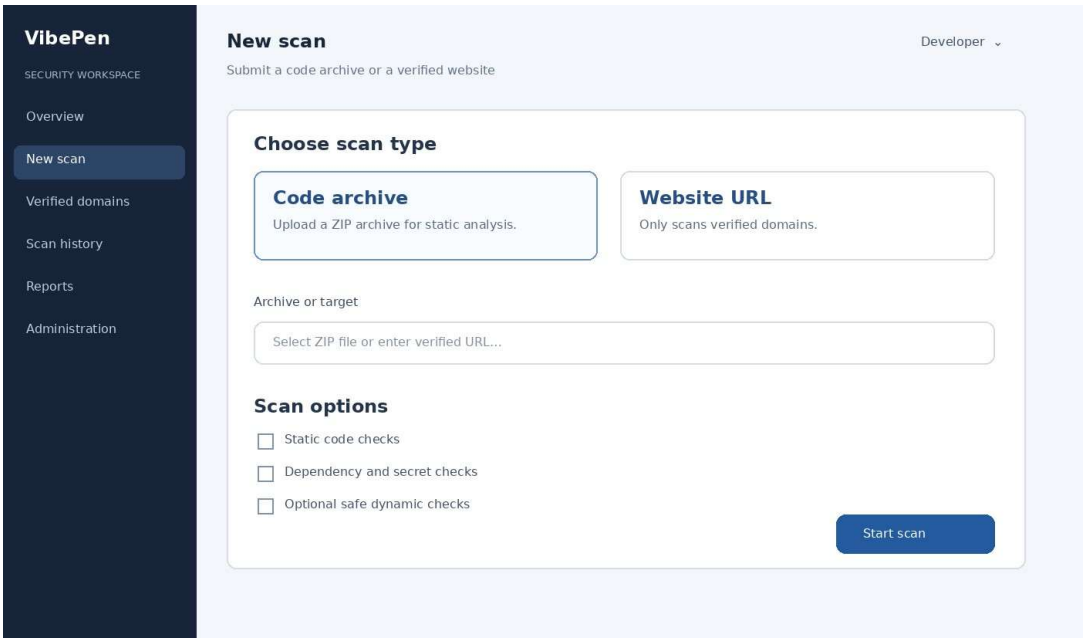


Figure 4.3: New scan screen for code and verified URL targets

Figure 4.3 separates the code and website workflows to make their different safety preconditions visible.

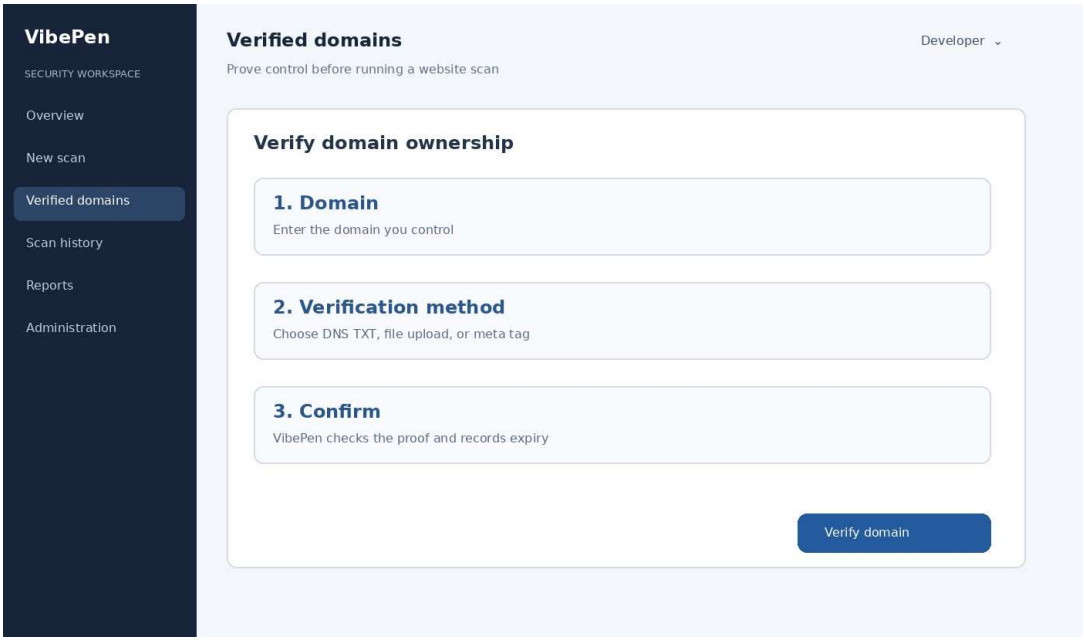


Figure 4.4: Domain ownership verification workflow

The verification steps shown in Figure 4.4 communicate that a website scan requires a valid proof of control.

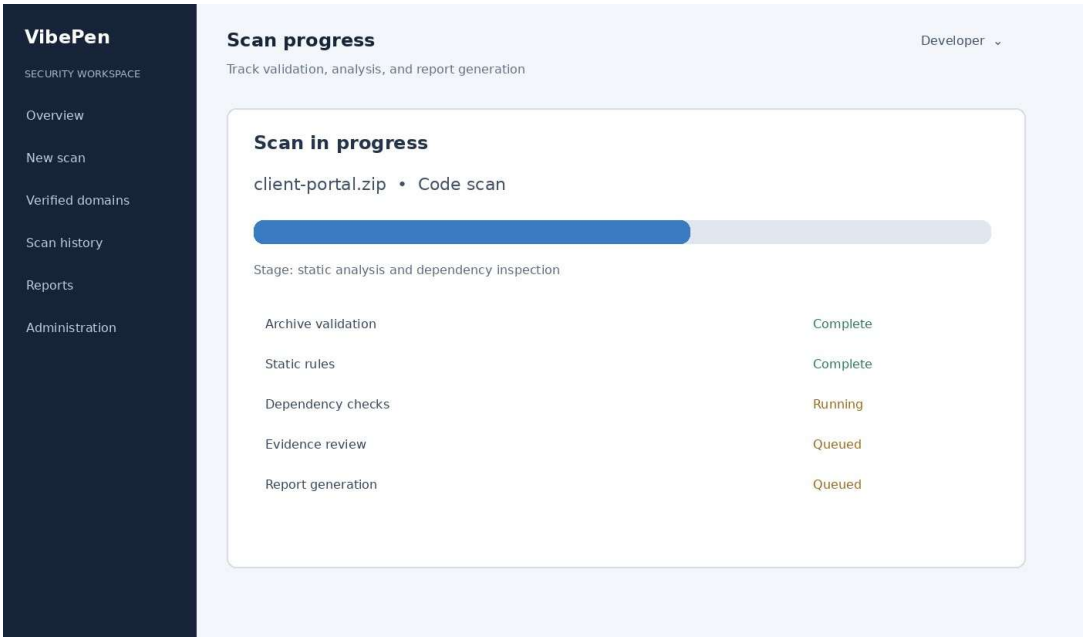


Figure 4.5: Scan progress and pipeline stages

Figure 4.5 makes completed, active, and queued stages distinguishable so users can identify incomplete

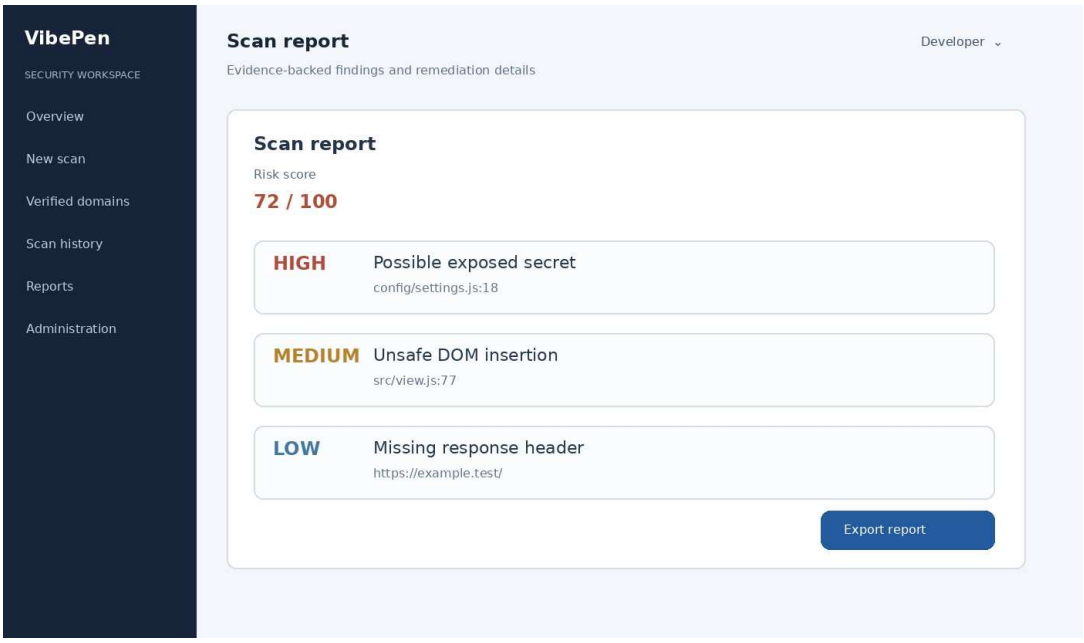


Figure 4.6: Report view with findings and export action

Figure 4.6 presents the information needed to triage a finding and locate its recommended correction.

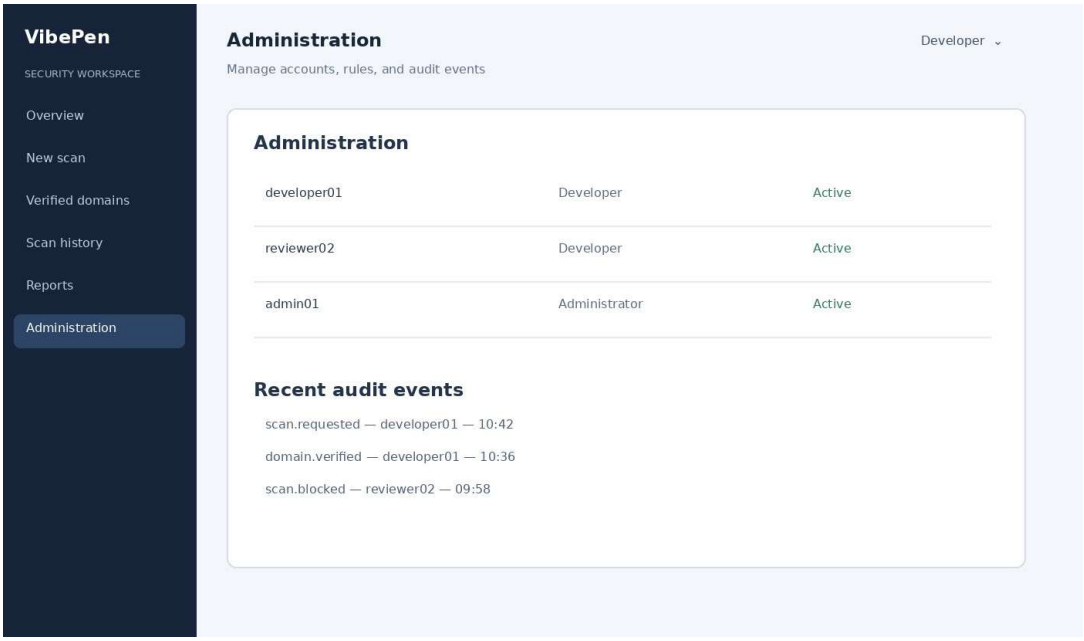


Figure 4.7: Administration view with accounts and audit events

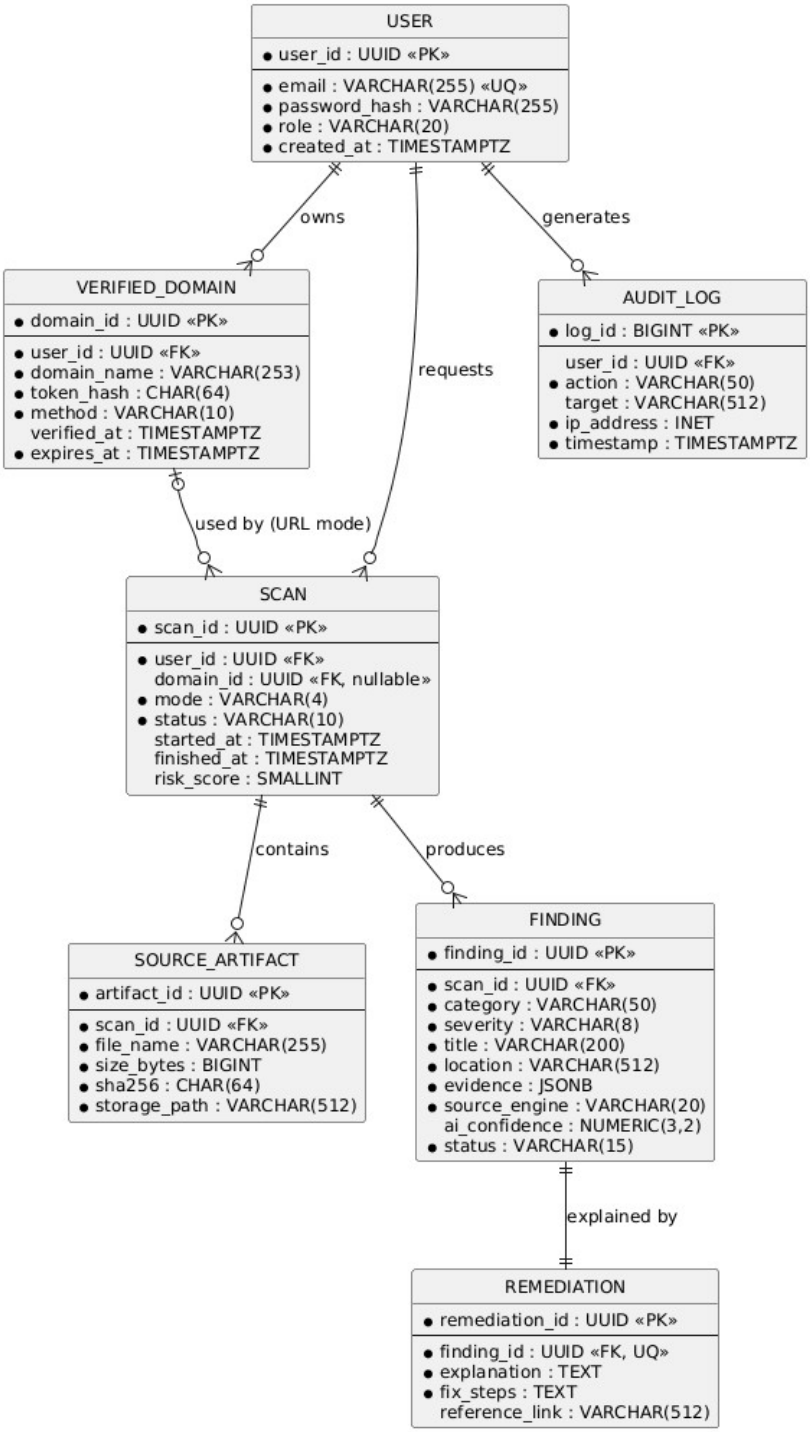
The administration view in Figure 4.7 groups privileged operations and audit information separately from ordinary scan activity.

4.9 Database Design

VibePen uses a relational database because users, verified domains, scans, artifacts, findings, remediation records, and audit events have structured relationships that benefit from foreign keys and transactional updates. PostgreSQL is a suitable candidate because its JSONB type can store redacted evidence whose shape varies between source-code excerpts and HTTP observations. Sensitive values should be minimized: passwords and verification tokens are represented by hashes, artifacts are identified by checksum and restricted storage path, and retained evidence is redacted before storage. The database schema does not replace application-level authorization or scan-scope checks; those controls must be enforced by the service layer. Figure 6.5 shows the proposed seven-entity model.

4.9.1 Entity-relationship diagram

As shown in Figure 6.5, a user can own multiple verified domains and scans and can generate audit events. A scan can reference a verified domain when it uses URL mode, while code mode may leave that reference empty. Each scan can contain multiple source artifacts and findings. Each finding is associated with one remediation record, allowing the explanation and corrective steps to be updated independently of the raw evidence. Foreign keys should prevent orphaned records, and application logic must reject URL scans whose verification has expired. The schema should be migrated and tested alongside the API to keep data constraints and authorization behavior consistent.



4.8 Proposed VibePen entity-relationship model

4.9.2 Data dictionary

Tables 4.7 to 4.13 define the core attributes. PK denotes primary key, FK foreign key, NN not null, and UQ unique. UUID identifiers are proposed for externally referenced objects; timestamp values should include time-zone information.

Table 4.7: Data dictionary for the user entity

Attribute	Type	Constraints	Description
user_id	UUID	PK	Unique user identifier
email	VARCHAR(255)	NN, UQ	Login email
password_hash	VARCHAR(255)	NN	Password hash, never plaintext
role	VARCHAR(20)	NN	Developer or administrator
created_at	TIMESTAMPTZ	NN	Account creation time

Table 4.8: Data dictionary for the verified domain entity

Attribute	Type	Constraints	Description
domain_id	UUID	PK	Verification identifier
user_id	UUID	FK, NN	Owner of the domain
domain_name	VARCHAR(253)	NN	Domain being verified
token_hash	CHAR(64)	NN	Hash of verification token
method	VARCHAR(10)	NN	DNS, FILE, or META
verified_at	TIMESTAMPTZ	–	Time proof was confirmed
expires_at	TIMESTAMPTZ	NN	Verification expiry

Table 4.9: Data dictionary for the scan entity

Attribute	Type	Constraints	Description
scan_id	UUID	PK	Unique scan identifier
user_id	UUID	FK, NN	Requesting user
domain_id	UUID	FK, nullable	Verified domain for URL mode
mode	VARCHAR(4)	NN	URL or CODE
status	VARCHAR(10)	NN	Queued, running, completed, failed
started_at	TIMESTAMPTZ	–	Worker start time
finished_at	TIMESTAMPTZ	–	Scan end time
risk_score	SMALLINT	0–100	Overall risk score

Table 4.10: Data dictionary for the source artifact entity

Attribute	Type	Constraints	Description
artifact_id	UUID	PK	Upload identifier
scan_id	UUID	FK, NN	Parent scan
file_name	VARCHAR(255)	NN	Original archive name
size_bytes	BIGINT	NN	Archive size in bytes
sha256	CHAR(64)	NN	Integrity checksum
storage_path	VARCHAR(512)	NN	Restricted storage location

Table 4.11: Data dictionary for the finding entity

Attribute	Type	Constraints	Description
finding_id	UUID	PK	Finding identifier
scan_id	UUID	FK, NN	Scan that produced the finding
category	VARCHAR(50)	NN	Weakness category
severity	VARCHAR(8)	NN	Critical, High, Medium, Low, Info
title	VARCHAR(200)	NN	Short finding name
location	VARCHAR(512)	NN	File/line or URL/parameter
evidence	JSONB	NN	Redacted supporting evidence
source_engine	VARCHAR(20)	NN	Static, dynamic, or both
ai_confidence	NUMERIC(3,2)	–	Reasoning confidence, if used
status	VARCHAR(15)	NN	Open, false positive, accepted, fixed

Table 4.12: Data dictionary for the remediation entity

Attribute	Type	Constraints	Description
remediation_id	UUID	PK	Remediation identifier
finding_id	UUID	FK, UQ, NN	Finding being explained
explanation	TEXT	NN	Problem and impact
fix_steps	TEXT	NN	Corrective steps
reference_link	VARCHAR(512)	–	Public weakness reference

Table 4.13: Data dictionary for the audit log entity

Attribute	Type	Constraints	Description
log_id	BIGINT	PK	Audit event identifier
user_id	UUID	FK	User responsible, when known
action	VARCHAR(50)	NN	Event type
target	VARCHAR(512)	–	Affected domain, scan, or account
ip_address	INET	NN	Request source address
timestamp	TIMESTAMPTZ	NN	Event time

4.10 Risk Analysis

The main risks concern the trustworthiness of automated explanations, hostile input, misuse against third-party systems, privacy exposure, and project delivery. A language model may overstate a candidate issue or follow malicious instructions embedded in scanned content; therefore, its output must be constrained to evidence supplied by the scanner and treated as advisory. Archives and URLs are untrusted inputs and require validation, isolation, scope enforcement, and rate limits. Secret redaction and data minimization reduce the chance of disclosing credentials to storage or an external service. The evaluation may also be limited by the availability of labeled applications, and model latency or service outages may affect completion time. Table 4.14 records the principal risks and mitigations, while Figure 4.9 presents an illustrative qualitative matrix.

Table 4.14: Project risks and mitigation measures

ID	Risk	Likelihood / impact	Mitigation
R1	Reasoning layer misjudges a finding	High / High	Evidence-bound prompts, schema validation, human review
R2	Prompt injection in scanned content	Medium / High	Treat content as data; provide no model tools; validate output
R3	Scan targets a third party	Medium / Very high	Ownership verification, scope guards, rate limits, audit logs
R4	Malicious archive or traversal	Medium / High	Safe extraction, size limits, no execution, isolation
R5	Server-side request forgery	Medium / High	Block private ranges and validate every redirect
R6	Dynamic checks disrupt a target	Low / High	Non-destructive checks, request caps, stop conditions
R7	Secrets disclosed externally	Medium / High	Redact secrets and send minimal context
R8	Insufficient labeled test cases	High / Medium	Build seeded test projects and document corpus limits
R9	Model cost, latency, or outage	Medium / Medium	Cache where safe and preserve deterministic results
R10	Scope exceeds project schedule	High / Medium	Prioritize high-value requirements and supported languages
R11	Tool updates alter results	Medium / Low	Pin versions and record them in every report

As Figure 4.9 indicates, risks involving unsafe scanning, hostile inputs, and evidence interpretation deserve early attention because their consequences can extend beyond the project team. The matrix is a planning aid rather than a measured probability model; the team should review it at each milestone and revise placements when implementation or testing provides new evidence. Risk controls should be verified through explicit test cases, including malformed archives, expired ownership proofs, redirect-to-private-address attempts, model output that lacks supporting evidence, and interrupted scanner workers.

ID	Risk	Likelihood	Impact	Rating	Mitigation	Contingency	Related req.
R1	Reasoning layer misjudges a finding	High	High	Critical	Evidence-bound prompts, schema validation, human review	Mark finding unvalidated; fall back to raw tool output	FR-23, FR-24, FR-33
R2	Prompt injection in scanned content	Medium	High	High	Treat content as data; no model tools; validate output	Discard output failing schema or evidence checks	FR-22, FR-24
R3	Scan targets a third party	Medium	Very high	Critical	Ownership verification, scope guards, rate limits, audit logs	Suspend account, block domain, review audit log	FR-08 to FR-12, FR-41
R4	Malicious archive or path traversal	Medium	High	High	Safe extraction, size limits, no execution, isolation	Reject archive, delete extracted files, log attempt	FR-05, FR-07
R5	Server-side request forgery	Medium	High	High	Block private ranges; validate every redirect	Abort scan, record blocked destination	FR-12
R6	Dynamic checks disrupt a target	Low	High	Medium	Non-destructive checks, request caps, stop conditions	Auto-stop on repeated errors; lower rate limit	FR-17, FR-18, NFR-07
R7	Secrets disclosed externally	Medium	High	High	Redact secrets; send minimal context	Purge stored evidence; disable external calls	FR-21, FR-22, NFR-05
R8	Insufficient labeled test cases	High	Medium	High	Build seeded test projects; document corpus limits	Report limits openly; add more seeded apps	NFR-08
R9	Model cost, latency, or outage	Medium	Medium	Medium	Cache where safe; preserve deterministic results	Continue with unvalidated findings; switch provider	Sec. 4.4, FR-33
R10	Scope exceeds project schedule	High	Medium	High	Prioritise high-value requirements and languages	Drop lowest-priority items; document as limitations	Sec. 4.11
R11	Tool updates alter results	Medium	Low	Low	Pin versions; record in every report	Re-run with recorded versions to compare	FR-40, NFR-12

Figure 4.9: Qualitative likelihood and impact matrix for project risks

4.11 Conclusion

This chapter has specified VibePen's main features, functional requirements, quality constraints, assumptions, use cases, interface concept, data model, and project risks. The requirements emphasize that code archives are inspected without execution, URL scans require valid ownership verification, and findings must retain traceable evidence. The proposed relational model separates users, domains, scans, source artifacts, findings, remediation records, and audit events. The interface mockups connect the principal workflows to the requirements, while the risk register identifies controls that need explicit implementation and evaluation. These requirements should now be mapped to architecture components and test cases. Any requirement that cannot be implemented within the project schedule should be documented as a limitation rather than silently removed from the specification.

Chapter 6 High-Level and Low-Level Design

This chapter presents the high-level and low-level design of VibePen. The high-level design describes the system architecture, technology stack, modules, data flow, and deployment structure. The low-level design explains the database, user interfaces, API endpoints, class structure, and key algorithms. The chapter also discusses the security measures and design decisions used to make the system reliable, modular, and secure.

6.1 System Overview

VibePen is an AI-assisted security analysis system designed to identify potential security vulnerabilities in web applications. It combines static analysis, dynamic analysis, and AI-based reasoning to detect issues, validate findings, and generate reports with explanations and suggested fixes.

The system supports two scanning modes. In code-based scanning, users upload a project archive for static analysis. In URL-based scanning, the system examines a live web application after verifying that the user is authorised to scan the target domain.

VibePen follows a modular, service-oriented architecture. The frontend provides an interface for users to manage scans and view reports, while the backend handles authentication, validation, scan scheduling, analysis, and report generation. Separate workers perform static and dynamic analysis in isolated environments. Their results are combined and processed by the AI reasoning module before being stored in the database.

The main objectives of the design are to ensure that only authorised targets are scanned, reduce false-positive findings, protect sensitive information, and provide understandable results to users. These objectives guide the architecture and the module boundaries described in the following sections. They also inform the decisions about data storage, scan isolation, evidence validation, and report generation. The design is intended to keep each stage understandable and independently testable, so that failures can be identified without losing all results from a scan.

6.1.1 Design Principles

The design of VibePen follows these principles:

- Authorisation first: Scan live domains only after ownership verification.
- Evidence-based analysis: AI explanations must be supported by tool evidence.
- Isolation: Process archives and targets in controlled workers with resource limits.

- Modularity: Use common interfaces so analysis engines can be replaced independently.
- Graceful degradation: Retain available findings and report incomplete checks after failures.
- Data minimisation: Redact secrets and share only the context required for AI analysis.

6.1.2 System Architecture

VibePen uses a layered architecture in which the frontend communicates with a REST API. The API validates requests, checks permissions, and places scan jobs in a queue. Isolated workers execute the appropriate analysis engines. The resulting findings are normalised, processed by the AI reasoning module, assigned severity levels, and stored in the database. Finally, the report generator creates a structured report that users can access through the dashboard.

The main layers of the system are summarised below. Figure 6.1 illustrates how the presentation, API, processing, AI reasoning, and data layers interact.

- Presentation layer: Provides authentication, scan setup, progress, and reports.
- API layer: Validates requests and handles authentication, permissions, and rate limits.
- Processing layer: Schedules scans and runs isolated analysis workers.
- AI reasoning layer: Validates findings and generates evidence-based explanations and fixes.
- Data layer: Stores users, domains, scans, findings, remediation details, and audit logs.

6.1.3 Technology Baseline

The proposed technology stack is selected to support modular development, efficient processing, and secure analysis. Table 6.1 summarises the technologies and their intended roles. These are proposed design choices rather than claims that every component has already been implemented or evaluated. The components communicate through defined interfaces, allowing individual technologies to be replaced when necessary.

6.1.4 Module Design

VibePen is divided into modules according to their responsibilities. This separation makes the system easier to maintain, test, and extend.

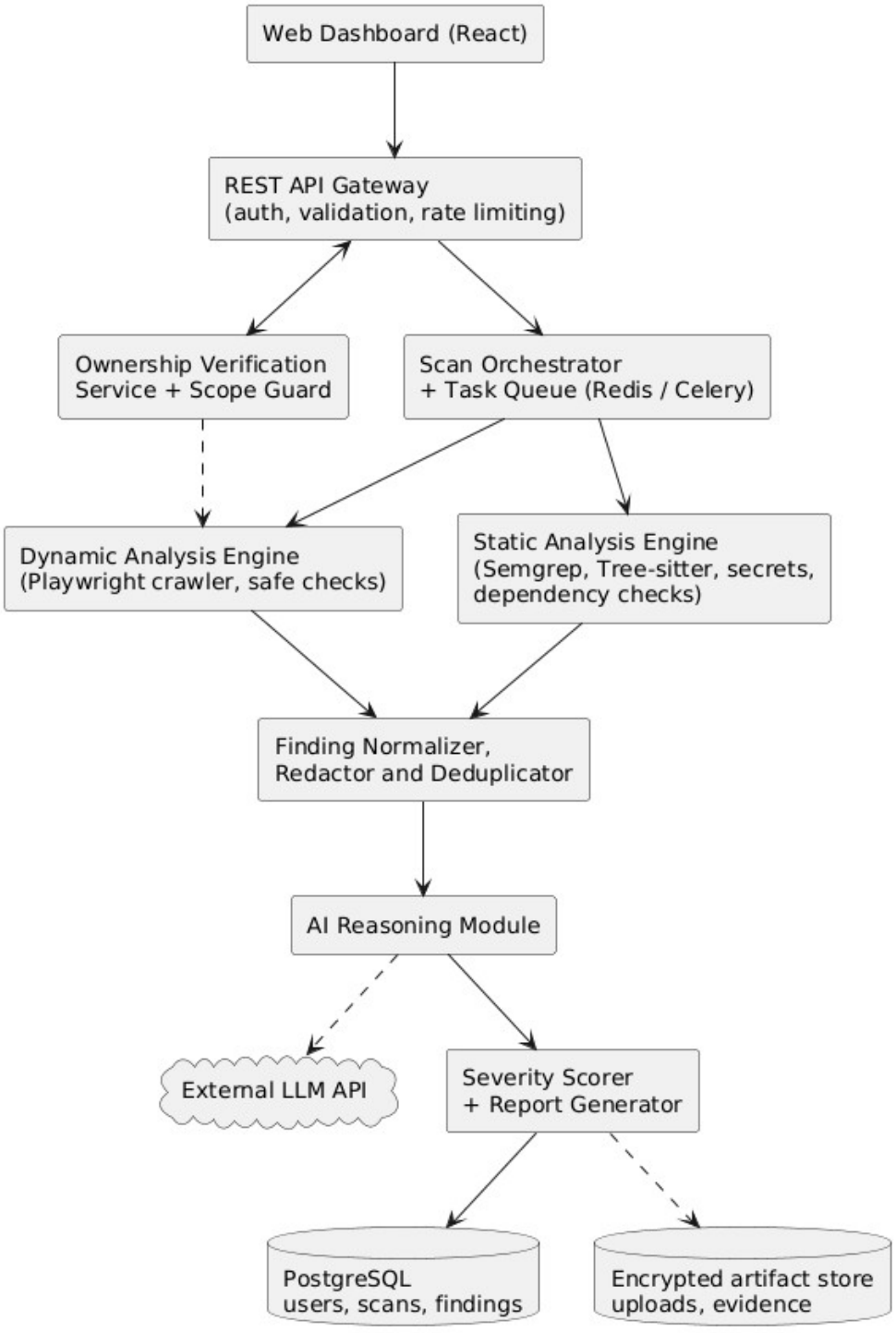


Figure 6.1: High-level architecture of VibePen

Table 6.1: Proposed technology stack

Layer	Technology	Reason for Selection
Frontend	React with a utility CSS framework	Supports reusable components and interactive dashboards.
API/Backend	Python with FastAPI	Provides typed request schemas and asynchronous request handling.

Job Queue	Celery with Redis	Supports background processing, parallel scans, and task retries.
Database	PostgreSQL	Provides relational integrity and support for structured and JSON data.
Static Analysis	Semgrep CLI, Tree-sitter, and custom rules	Supports code analysis and extensible security checks.
Dynamic Analysis	Playwright with Chromium in headless mode	Supports automated browsing of client-rendered web applications.
Isolation	Docker containers with resource limits	Separates analysis tasks and limits the impact of untrusted input.
LLM Access	Provider-independent adapter to a hosted LLM API	Allows the language model provider to be changed without redesigning the system.
Report Generation	HTML templates rendered to PDF, with JSON export	Provides readable reports and machine-readable output.

6.1.4.1 Web Dashboard and API Gateway

The web dashboard provides the user interface for account management, scan creation, scan progress, and report viewing. The API gateway receives requests from the frontend and validates them before processing. It also handles token-based authentication, per-user rate limits, and audit logging. It acts as the only component directly accessible from the internet.

6.1.4.2 Ownership Verification Service and Scope Guard

The ownership verification service confirms that a user is authorised to scan a live domain. It generates a random verification token and stores its hash with the associated user, domain, verification method, and expiry time.

Users can verify a domain through one of three methods. The verification flow is shown in Figure 6.2.

- Adding a TXT record at `vibepen-verify.<domain>`.
- Placing a verification file at `/.well-known/vibepen-verify.txt`.
- Adding a verification meta tag to the website's home page.

A successful verification remains valid for 30 days. Subdomains are verified separately unless the parent domain is verified and the user explicitly chooses to include them.

The Scope Guard controls outgoing requests made by the dynamic analysis engine. Before a request is sent, it checks whether the destination belongs to an authorised domain, resolves the hostname, and rejects private, loopback, or link-local IP addresses. Redirects are checked again before following them. These controls help prevent unauthorised scanning and server-side request forgery (SSRF), a risk addressed in the OWASP prevention guidance [17].

6.1.4.3 Static Analysis Engine

The static analysis engine examines uploaded project archives without executing their code. It uses established tools such as Semgrep and Tree-sitter for rule-based checks and source-code structure analysis [4, 5]. It operates inside a network-isolated container and consists of the following components:

- Safe extractor: Enforces archive size, file-count, and path limits.
- Rule scanner: Uses Semgrep and custom rules to detect secrets and common insecure code patterns.
- Code mapper: Uses Tree-sitter to map routes, middleware, calls, and database operations.
- Dependency checker: Reviews manifests and flags missing, new, or look-alike package names.

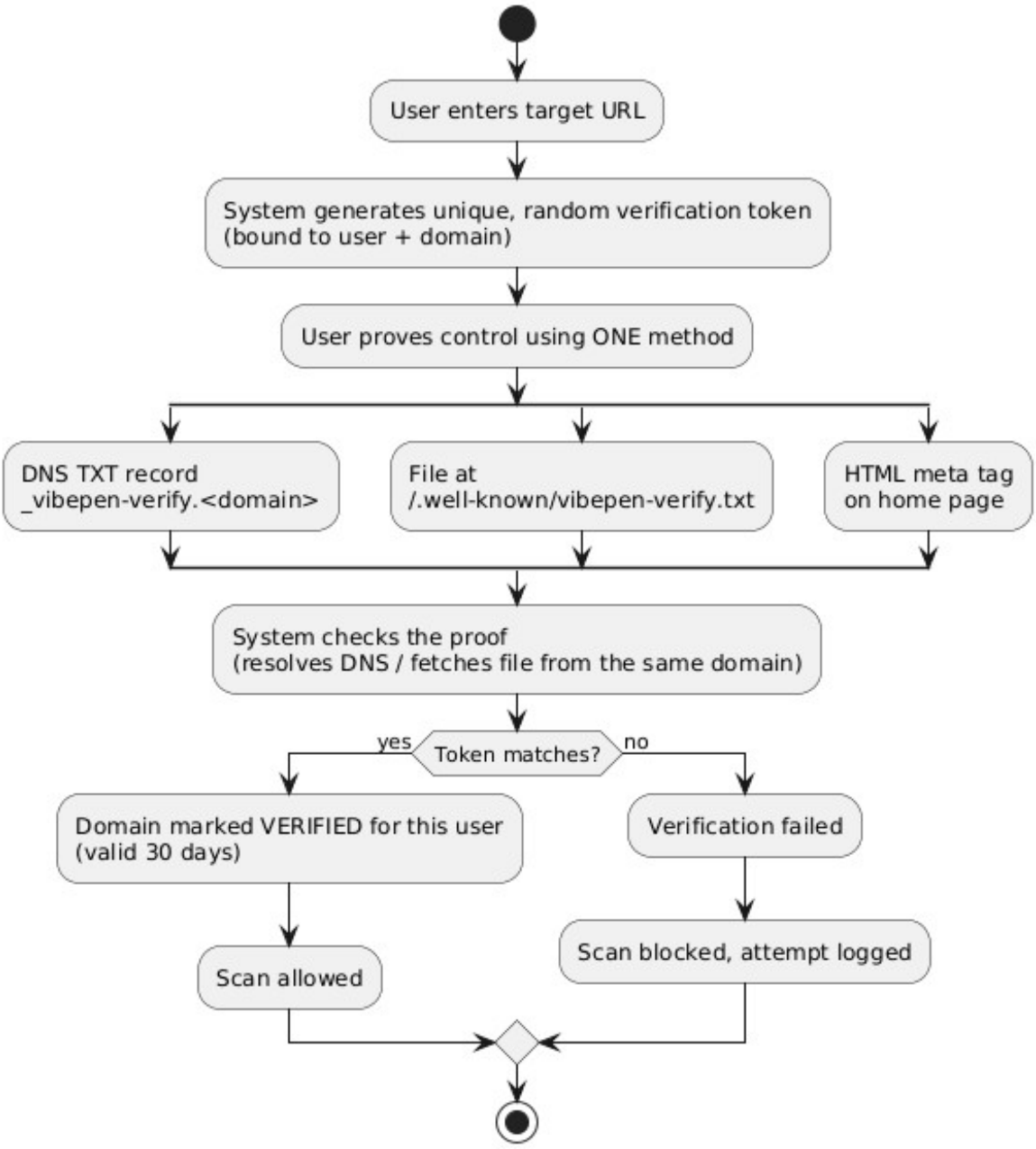


Figure 6.2: Ownership verification flow

6.1.4.4 Dynamic Analysis Engine

The dynamic analysis engine uses Playwright to browse a live web application within configured page, depth, time, and request-rate limits [6]. All outgoing requests are controlled by the Scope Guard.

The engine performs non-destructive checks, including:

- Passive checks: Review headers, cookie flags, CORS, exposed paths, and response content.
- Backend access checks: Use read-only requests to check access controls when a public client key is found.
- Probe checks: Use harmless markers to identify reflected input and database errors.
- IDOR checks: Optionally uses two test accounts supplied by the owner to check whether one account can access another account's objects.

The engine stops early if the target repeatedly returns server errors or times out. This helps reduce unnecessary requests and limits the impact on the target application.

6.1.4.5 Finding Normalizer and AI Reasoning Module

The finding normalizer converts results from different analysis engines into a common schema. Each finding contains its category, location, evidence, source engine, and initial severity. Duplicate findings are merged, and sensitive values are redacted before further processing.

The AI reasoning module performs three main tasks:

- Triage: Classifies findings and records confidence with supporting evidence.
- Linking: Connects related routes, middleware, and database operations.
- Explanation: Describes impact and recommends remediation in plain language.

Several safeguards are applied to the AI reasoning process. Outputs must follow a JSON schema, and invalid outputs are retried once before being discarded. Evidence references are checked against the supplied context. The model cannot create findings without supporting tool evidence. Scanned content is treated as data, the model is not provided with tools or network access, and secret values are replaced with placeholders before external processing. If the model is unavailable, findings are retained and marked as unvalidated.

6.1.4.6 Severity Scoring

Each finding is assigned a severity based on its vulnerability category, exploitability evidence, and exposure. The overall risk score is calculated using finding weights, capped at 100, and presented with a letter grade.

Table 6.2: Severity classification rules

Severity	Criteria	Example
Critical	Directly exploitable with severe impact, such as exposed live secrets or unauthorised access to private data.	A service-role database key exposed in client-side code.
High	Likely to be exploitable with significant impact, possibly requiring only a simple additional step.	A confirmed IDOR vulnerability or an injectable parameter.
Medium	A weakness that requires additional conditions before exploitation.	Wildcard CORS on an API that uses cookies.
Low	A security hardening gap with limited direct impact.	A missing security header.
Informational	An observation that provides useful information but is not itself a vulnerability.	Disclosure of a framework version.

The risk score uses the following weights: Critical = 25, High = 10, Medium = 4, and Low = 1. The sum of the weights is capped at 100. Informational findings do not contribute to this weighted score.

6.1.4.7 Report Generator

The report generator creates a structured report containing the overall risk score, counts of findings by severity, and a prioritised list of issues. Table 6.2 defines the proposed severity categories and their illustrative criteria. Each finding includes its explanation, possible impact, redacted evidence, location, confidence, and suggested fixes.

The report also records tool versions, scan settings, and checks that could not be completed. Users can view reports through the dashboard or export them in PDF and JSON formats.

6.1.5 Data Flow and Interaction

The scan process begins when a user selects a scanning mode and submits the required input. Figure 6.3 presents the main data flow, and Figure 6.4 shows the interactions between the components. For URL-based scans, the system checks domain ownership before allowing the scan to proceed. For code-based scans, the uploaded archive is validated and prepared for static analysis.

The job queue assigns the scan to the appropriate worker. The analysis engines generate findings, which are normalised, redacted, and deduplicated. The AI reasoning module then evaluates and links the

evidence. The system calculates severity levels, generates the report, stores the results, and updates the scan status.

Figure 6.3 represents the movement of data through the system, while Figure 6.4 represents the interactions between the main components during a typical scan.

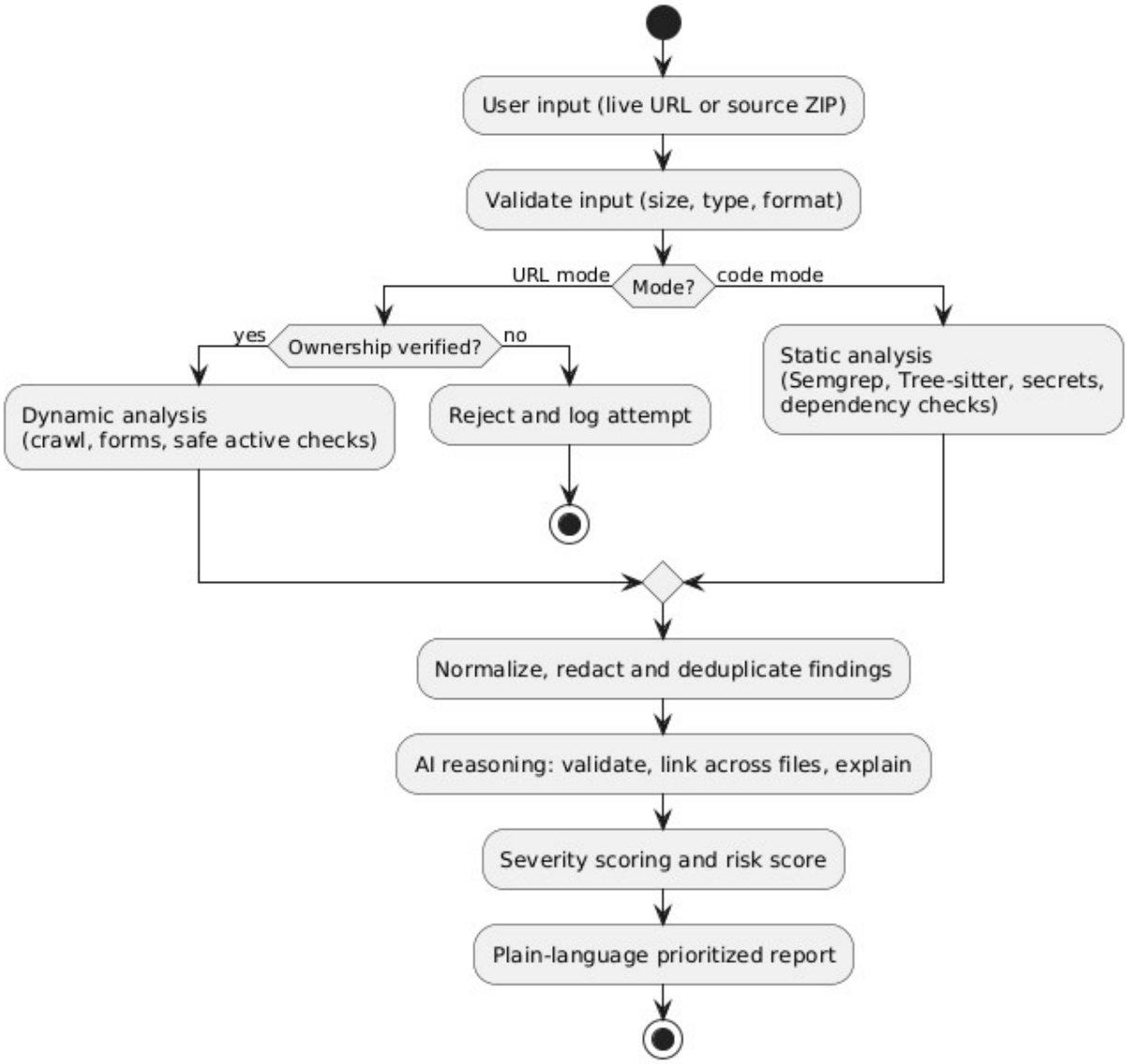


Figure 6.3: Data flow of a VibePen scan

6.1.6 Deployment View

VibePen is designed to run its main components in separate containers. The API, task queue, workers, and database are deployed independently so that their resources and responsibilities can be managed separately.

Static and dynamic analysis workers use different container images. The static analysis worker has no outbound network access, while the dynamic analysis worker can send requests only through the Scope Guard. Uploaded archives and evidence are stored in an encrypted volume and removed according to the defined retention period.

This deployment structure improves isolation, supports parallel processing, and reduces the risk that a failure in one component affects the entire system.

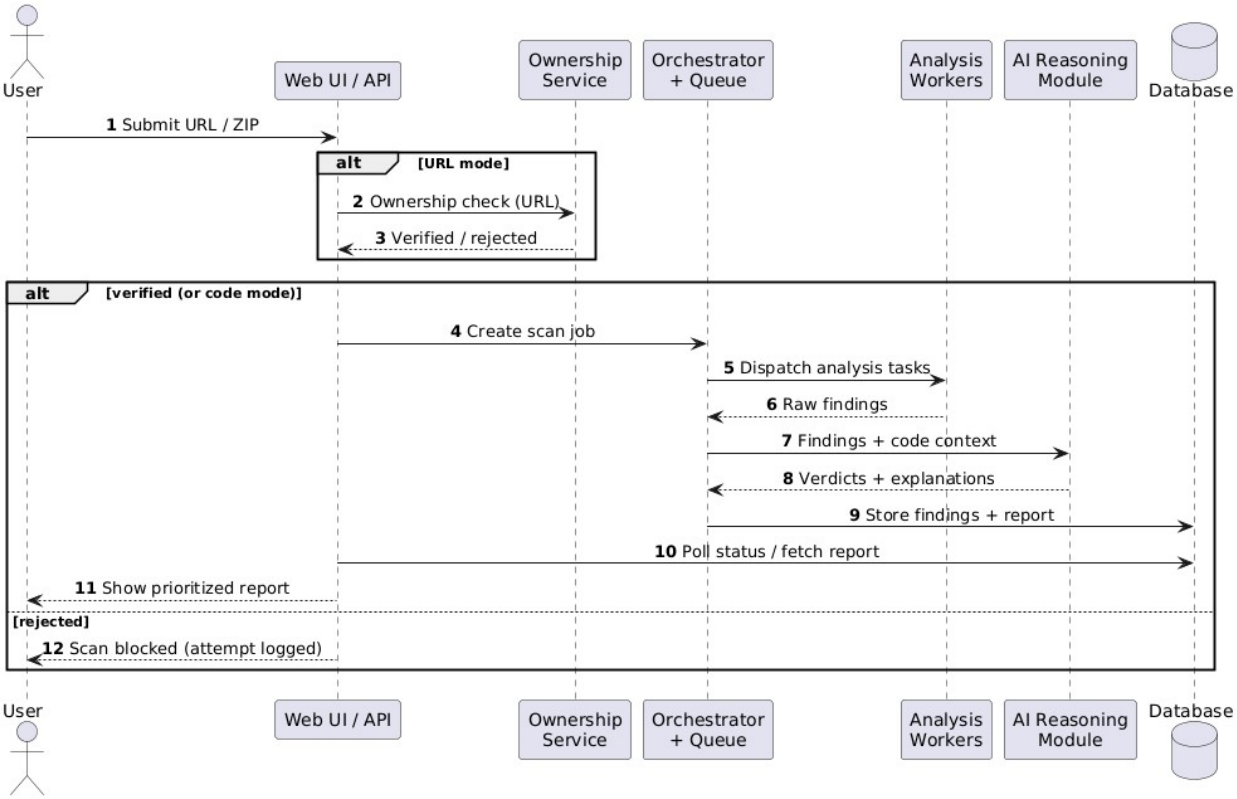


Figure 6.4: Sequence diagram of a scan

6.2 Design Considerations

Several design considerations must be addressed to ensure that VibePen operates reliably and securely. These include database organisation, user-interface usability, access control, safe analysis of untrusted inputs, and the selection of appropriate algorithms and interfaces.

6.2.1 Database Design

VibePen uses PostgreSQL to store user accounts, verified domains, scan information, uploaded source artifacts, findings, remediation details, and audit logs. The database is designed to maintain relationships between these records and to ensure that each scan and its findings remain associated with the appropriate user.

Figure 6.5 shows the proposed entity-relationship model.

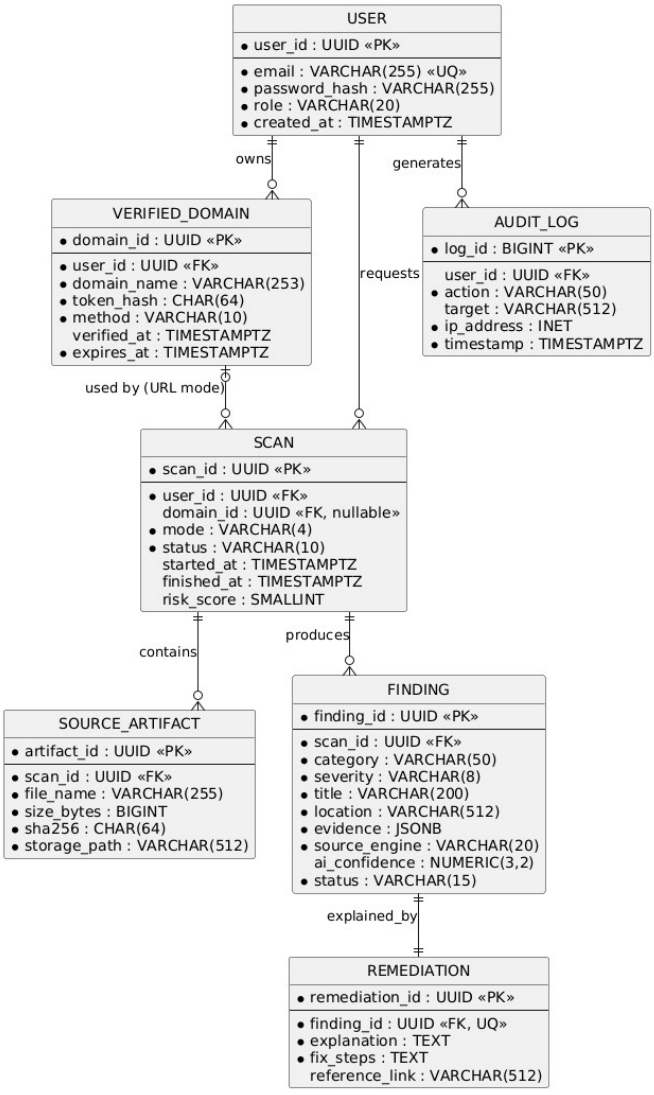


Figure 6.5: Architecture and class diagram of VibePen

The main entities and their design considerations are described in Table 6.3.

Table 6.3: Key database design considerations

Entity	Design Considerations
USER	Contains account information and a role field for role-based access control. Passwords are stored as hashes using a modern adaptive hashing algorithm.
VERIFIED_DOMAIN	Stores the domain, verification method, token hash, and expiry time. URL-based scans must reference a valid, unexpired verification record.
SCAN	Stores the scanning mode, target, status, and risk score. The status follows the defined lifecycle: queued, running, completed, or failed.
SOURCE_ARTIFACT	Stores information about an uploaded project archive. A SHA-256 hash supports integrity checks and duplicate detection.
FINDING	Stores the category, location, severity, status, and redacted evidence in JSON format. Its status can indicate a false positive, accepted issue, or fixed issue.
REMEDIATION	Stores the explanation and recommended fix steps associated with a finding.
AUDIT_LOG	Records scans, ownership checks, blocked attempts, and administrative actions. The log is designed to be append-only.

6.2.2 User Interface Design

The user interface is designed to make the scanning process understandable to both technical and non-technical users. It provides clear navigation, visible scan statuses, and reports that explain each finding and how it can be addressed.

The main screens are described in Table 6.4.

Table 6.4: Planned user-interface screens

Screen	Purpose and Main Elements
Login / Register	Allows users to create accounts and sign in.
Dashboard	Displays scan history, status indicators, risk-score trends, and a button to start a new scan.
New Scan	Allows users to select code-based or URL-based scanning, upload an archive or enter a URL, and configure optional test accounts and limits.
Ownership Wizard	Displays the verification token, instructions for the available verification methods, and the verification result.
Scan Progress	Shows the current processing stage and any warnings or incomplete checks.
Report	Displays the overall score, severity counts, filtering and sorting options, finding details, evidence, remediation steps, and export options.
Admin	Provides an interface for managing users, viewing audit logs, and configuring limits and rule sets.

The interface follows three main guidelines. First, information is presented in plain language wherever possible. Second, each finding explains what the problem is, why it matters, and how it can be fixed. Third, severity is indicated through labels and icons as well as colour, while detailed technical information can be expanded when required.

6.2.3 Security Design

Security is a major consideration because VibePen analyses source code and live web applications, both of which may contain untrusted input. The security controls are designed around common web application risks and should be reviewed against recognised guidance such as the OWASP Top 10 [2]. The system therefore uses preventive controls at multiple levels.

Table 6.5: Security threats and design responses

Threat	Design Response
Scanning an unauthorised website	Requires token-based domain ownership verification, Scope Guard checks, audit logging, and account suspension when necessary.
SSRF through URLs or redirects	Resolves hostnames, checks IP address ranges, blocks internal addresses, and re-evaluates redirects.
Malicious uploaded archive	Applies safe extraction rules, file-size and file-count limits, and isolated containers without network access. Uploaded code is not executed by the static analysis engine.
Prompt injection through scanned content	Treats scanned content as data, limits model capabilities, validates structured outputs, and verifies evidence references.
Exposure of discovered secrets	Redacts secret values before storage and AI processing, uses encrypted storage, and applies retention limits.
Harm to the target website	Uses read-only checks and harmless probes, applies request-rate limits, and stops scans when repeated errors or timeouts occur.
Account abuse	Uses password hashing, authentication, rate limiting, per-user quotas, and role-based access control.
Tampering with audit records	Uses an append-only audit-log design with restricted write access.

6.2.4 Class Design

The scanning service follows an interface-based design. The main orchestrator coordinates the scan and communicates with analysis engines through a common `Analyzer` interface. This allows different implementations, such as `StaticAnalyzer` and `DynamicAnalyzer`, to be used without changing the main orchestration logic.

Figure 6.6 presents the proposed class structure.

The main classes and their responsibilities are as follows:

- ScanOrchestrator: Coordinates the scan workflow and analyzers.
- Analyzer: Defines the shared analysis-engine interface.
- StaticAnalyzer: Analyses source code and returns potential findings.
- DynamicAnalyzer: Checks authorised web applications through controlled browser requests.
- FindingNormalizer: Standardises findings, removes duplicates, and redacts secrets.

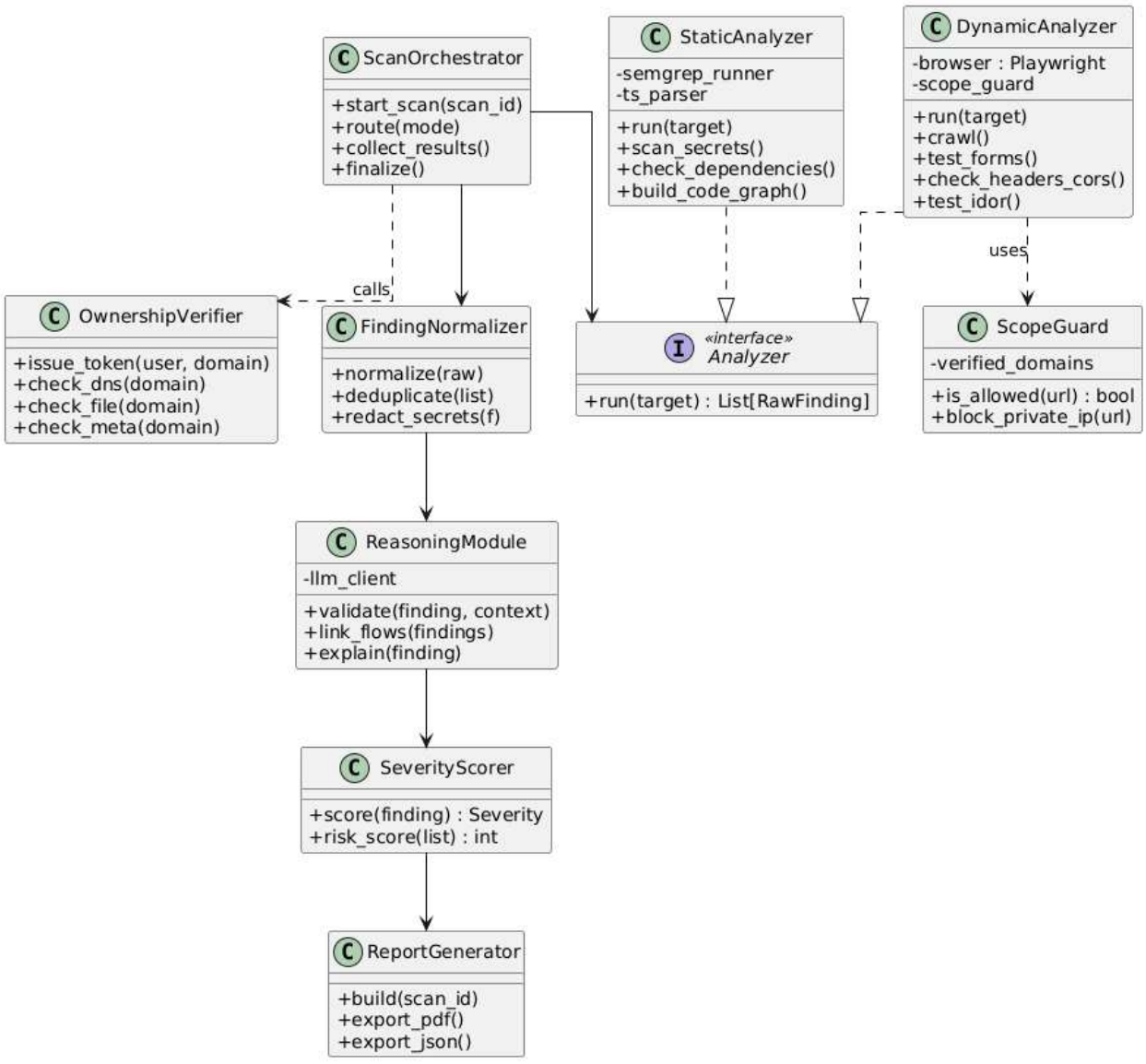


Figure 6.6: Class diagram of the scanning service

- AIReasoningModule: Evaluates evidence and explains related findings.
- SeverityScorer: Assigns severity and calculates the risk score.
- ReportGenerator: Exports reports in supported formats.

6.2.5 REST API Design

The REST API provides communication between the frontend and backend. Table 6.6 lists the proposed endpoints and their intended functions. Each endpoint performs a specific operation and checks the user's permissions before accessing or modifying protected resources.

Table 6.6 lists the main API endpoints.

Table 6.6: Main REST API endpoints

Method and Path	Purpose
POST /api/auth/register	Creates a user account.
POST /api/auth/login	Authenticates a user and returns a session token.
POST /api/domains	Starts ownership verification and returns a token with verification instructions.
POST /api/domains/{id}/verify	Checks ownership proof and updates the domain's verification status.
POST /api/scans	Creates a code-based or URL-based scan after validating the request and required authorisation.
GET /api/scans	Lists scans accessible to the requesting user.
GET /api/scans/{id}	Returns the status and summary of a particular scan.
GET /api/scans/{id}/findings	Returns findings for a scan, with supported filters.
PATCH /api/findings/{id}	Updates a finding's status, such as false positive, accepted, or fixed.
GET /api/scans/{id}/report?format=pdf	Downloads a report in PDF format.
GET /api/scans/{id}/report?format=json	Downloads a report in JSON format.
GET /api/admin/audit-logs	Allows authorised administrators to view audit logs.

The API validates input and applies authentication and authorisation checks. Requests for scans of live websites are rejected if the required domain verification is missing or expired. Administrative endpoints are restricted to authorised administrators.

6.2.6 Key Algorithms

This subsection describes the main algorithms used to coordinate scanning, control outgoing requests, check dependencies, and remove duplicate findings.

6.2.6.1 Scan Pipeline

The scan orchestrator coordinates the complete analysis process. It checks authorisation where required, runs the applicable analyzers, processes their results, and generates the final report.

```
function run_scan(scan):
    if scan.mode == URL and not verified(scan.user, scan.domain):
        fail(scan, "domain not verified")
        return

    raw_findings = []

    for analyzer in analyzers_for(scan.mode):
        try:
            raw_findings += analyzer.run(scan.target)
        except Error as error:
            record_failed_check(scan, analyzer, error)

    findings = normalize(raw_findings)
    findings = redact(findings)
    findings = deduplicate(findings)

    findings = reasoning.validate_and_link(findings)

    for finding in findings:
        finding.severity = score(finding)

    report = build_report(scan, findings)
    save(scan, findings, report)
    mark_completed(scan)
```

If an analyzer fails, the system records the failure and retains the results produced by the other analyzers. This supports partial reporting instead of losing all findings because of one failed component.

6.2.6.2 Scope Verification Algorithm

Before every outgoing request, the Scope Guard verifies that the destination is permitted. It checks the hostname against the verified hosts and rejects addresses that belong to private, loopback, or link-local ranges.

```
function allowed(url, scan):
```

```
    host = url.host

    if host not in scan.verified_hosts:
        return False

    ip = resolve(host)

    if is_private(ip) or is_loopback(ip) or is_link_local(ip):
        return False

    return True

# Every redirect must be checked again.
```

This check is repeated for redirects to prevent a permitted URL from redirecting the crawler to an unauthorised destination.

6.2.6.3 Dependency Existence Check

The dependency checker reads package names from project manifests and queries the package registry through a proxy. A missing package is flagged for review, while newly published or suspiciously similar package names are marked for further investigation.

```
for package in manifest.dependencies:
    info = registry.lookup(package.name)

    if info is None:
        flag(Critical, "package does not exist")
    elif info.age_days < 30 or near_popular_name(package.name):
        flag(Medium, "new or look-alike package")
```

These checks use package existence, package age, and name similarity as indicators. The results are treated as potential security findings and should be interpreted with the supporting evidence.

6.2.6.4 Finding Deduplication

Two findings are considered duplicates when they have the same category, location, and evidence hash. For code findings, the location includes the file and line range. For URL findings, it includes the URL and parameter.

When duplicate findings are identified, they are merged and their source engines are recorded. If multiple engines identify the same issue, their agreement can increase confidence in the finding.

6.2.7 Design Decisions and Alternatives

Several alternatives were considered while designing VibePen. Table 6.7 summarises the principal choices and the reasons for selecting them. The selected approaches aim to balance detection capability, security, maintainability, and operational efficiency.

Table 6.7: Key design decisions and alternatives

Decision	Alternative Considered	Reason for Choice
Combine static analysis, dynamic analysis, and AI reasoning	Use a single analysis technique	Different techniques identify different types of weaknesses, while AI reasoning helps interpret and connect the evidence.
Use the LLM to validate and explain evidence	Allow the LLM to identify vulnerabilities without tool evidence	Evidence-based reasoning reduces unsupported findings and makes results easier to audit.
Verify domain ownership before live scans	Allow open scanning with a disclaimer	Ownership verification provides a technical control against unauthorised scanning.
Use non-destructive checks	Perform full active exploitation	Harmless checks reduce the risk of damaging target systems while still supporting vulnerability detection.
Do not execute uploaded code during static analysis	Run uploaded projects to test them	Executing unknown code creates additional security risks. Live dynamic analysis is handled separately through verified targets.
Use a provider-independent LLM adapter	Hard-code a single model provider	An adapter allows model changes and supports cost management and alternative configurations.
Use a task queue and isolated workers	Perform scans directly inside API requests	Background workers support parallel processing, time limits, retries, and better failure isolation.

6.3 Conclusion

This chapter presented the high-level and low-level design of VibePen. It described the system architecture, technology stack, responsibilities of the main modules, data flow, deployment structure, database design, user interface, security controls, class structure, API endpoints, and key algorithms.

The design combines static and dynamic analysis with evidence-based AI reasoning to produce structured security findings and remediation reports. Ownership verification, isolated workers, input validation, evidence checks, and secret redaction are included to reduce security risks. The modular architecture also allows individual components to be maintained or extended independently.

Bibliography

[1] Y. Fu, P. Liang, A. Tahir, Z. Li, M. Shahin, J. Yu, and J. Chen, "Security weaknesses of copilot-generated code in github projects: An empirical study," 2023.

[2] OWASP Foundation, "Owasp top 10: The ten most critical web application security risks," 2021. Accessed 9 October 2026.

[3] MITRE, "Common weakness enumeration." Accessed 9 October 2026.

[4] Semgrep, "Semgrep documentation." Available: https://semgrep.dev/docs/, 2026.

[5] Tree-sitter, "Tree-sitter documentation." Available: https://tree-sitter.github.io/tree-sitter/, 2026.

[6] Microsoft, "Playwright documentation." Available: https://playwright.dev/docs/, 2026.

[7] GitHub, "Codeql documentation." Available: https://codeql.github.com/docs/, 2026.

[8] SonarSource, "Sonarqube documentation." Available: https://docs.sonarsource.com/sonarqube/, 2026.

[9] PyCQA, "Bandit documentation," 2026. Accessed 9 October 2026.

[10] Gitleaks, "Gitleaks documentation." Available: https://gitleaks.org/, 2026.

[11] Snyk, "Snyk documentation." Available: https://docs.snyk.io/, 2026.

[12] OWASP Foundation, "Owasp dependency-check," 2026. Accessed 9 October 2026.

[13] Aqua Security, "Trivy documentation: Vulnerability scanning," 2026. Accessed 9 October 2026.

[14] OWASP, "Owasp zap documentation." Available: https://www.zaproxy.org/docs/, 2026.

[15] PortSwigger, "Burp suite documentation." Available: https://portswigger.net/burp/documentation, 2026.

[16] ProjectDiscovery, "Nuclei documentation." Available: https://docs.projectdiscovery.io/tools/nuclei/, 2026.

[17] OWASP Foundation, "Server side request forgery prevention cheat sheet." Accessed 9 October 2026.

[18] OWASP, "Owasp juice shop." Available: https://owasp.org/www-project-juice-shop/, 2026.

[19] Damn Vulnerable Web Application, "Damn vulnerable web application." Available: https://github.com/digininja/DVWA, 2026.

[20] OWASP, "Owasp webgoat." Available: https://owasp.org/www-project-webgoat/, 2026.

[21] PostgreSQL Global Development Group, "Postgresql documentation: Json types." Accessed 9 October 2026.

[22] R. O. Duda, P. E. Hart, and D. G. Stork, *Pattern Classification*. John Wiley and Sons, 2000.

[23] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[24] C.-C. Chang and C.-J. Lin, *LIBSVM: A Library for Support Vector Machines*, 2001. Available: http://www.csie.ntu.edu.tw/~cjlin/libsvm.

[25] I. Guyon, A. Saffari, G. Dror, and G. Cawley, "Agnostic learning vs. prior knowledge challenge," in *Proceedings of International Joint Conference on Neural Networks*, August 2007.

[26] "Agnostic Learning Vs. Prior Knowledge Challenge by Internation Joint Conference on Neural Networks (IJCNN)." Available: http://www.agnostic.inf.ethz.ch [Accessed November 02, 2018].

[27] I. Guyon, A. Saffari, G. Dror, and G. Cawley, "Analysis of the IJCNN 2007 agnostic learning vs. prior knowledge challenge," *Neural Networks*, vol. 21, pp. 544–550, December 2008.

Chapter A First Appendix if Required

If you want to add appendices then make sure you provide a proper title. Add different appendices by using the chapter command as shown for this one. They will automatically be added to the table of contents.

A.1 References

See how .bib file is prepared. The bibliography would be automatically added by Latex. So here is how you cite papers (you'll need to view the .bib file to understand). [22] is the correct way to make one citation and [23, 24] to make multiple citations. You can see various reference styles for manuals [24], conferences [25], websites [26] and journal articles [27]. Also, note that all citations appear in the order in which they appear within text. They are not sorted according to author names.

A.2 Equations

Make sure you use the math environment for writing equations. Again, make sure you do not hard code equation numbers. Use label and ref commands of Latex to number them and reference them. Here is an example of (A.1)

$$E = mc^2 \tag{A.1}$$

Note, how the equation is referenced by its equation number within brackets.

A.3 Figures

To insert a figure, you can use the following code as template. Here the figure is referenced using ref command of Latex. So Figure A.1 is the FAST logo. DO NOT HARD CODE FIGURE NUMBERS. Let Latex manage them for you.



Figure A.1: Fast Logo is the Caption.

A.4 Tables

Note how tables can be added in Latex. Also, make sure you use label and ref commands to define and use table numbers. You can use Table A.1 as a template to make tables, add captions to them and reference them. DO NOT HARD CODE TABLE NUMBERS. Let Latex manage their numbering for you.

If you are having problems in managing Latex tables, you may use Lyx https://www.lyx.org/.

Table A.1: Give a Caption to the Table.

No.	Column 1	Column 2
1.	Some data	More data
2.	Some values	More values

A.5 Pseudo Code

When you need to explain an existing algorithm of an algorithm you devised, then you should use pseudo code notation. Given below, algorithm 1, is an example. Note how the numbering is managed by the Latex itself.

Data: this text

Result: how to write algorithm with LaTeX2e

initialization;

while *not at end of this document* **do**

    read current;

    if *understand* **then**

        go to next section;

        current section becomes this one;

    else

        go back to the beginning of current section;

    end

end

Algorithm 1: How to write algorithms

A.6 Code of Programming Languages

If you need to add C++, Java or any other programming language's code, then you can use "listings" package. Following is an example. Normally, the code is only added in the appendices to avoid clutter

in the document.

```
#include <stdio.h>
#define N 10
/* Block
 * comment */

int main()
{
    int i;

    // Line comment.
    puts("Hello world!");

    for (i = 0; i < N; i++)
    {
        puts("LaTeX is also great for programmers!");
    }

    return 0;
}
```

A.7 Recommendations for LaTeX

Here we are providing you with only two Latex source files. main.tex and fypbib.bib file. You can also start a project at overleaf.com and upload the zip file of this template there and share the Latex source between your group and supervisor. This way all of you can work on the Latex files together online.